

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 1. oldal
--	--	--

Mikrokontroller alapú rendszerek

Elektronikus jegyzet

2. fejezet (3. rész)

Készítette: Szabó Zoltán mérnökstanár
BME Automatizálási és Alkalmazott Informatikai Tanszék
1117. Budapest, Magyar tudósok körútja 2.
Q épület B szárny II. em. QB218.
Tel: 463-2597
Fax: 463-2871 (adm.)
Mail: zszabo@aut.bme.hu

Hallgatják: Villamosmérnöki és Informatikai Kar
Nappali tagozat
Villamosmérnöki alapszak (BSc)
III. évfolyam, 5. félév
Beágyazott és irányító rendszerek specializáció hallgatói

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 2. oldal
--	--	--

COPYRIGHT

Jelen dokumentum a BME Villamosmérnöki és Informatikai Kar hallgatói számára készített elektronikus jegyzet. A dokumentumot a Mikrokontroller alapú rendszerek c. tárgyat (BMEVIAUAC06) felvevő hallgatók jogosultak használni, és saját céljukra 1 példányban kinyomtatni. A dokumentum módosítása, bármilyen eljárással részben vagy egészben történő másolása tilos, illetve csak a szerző előzetes engedélyével történhet.

Copyright © 2008-2023 / Szabó Zoltán

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 3. oldal
--	--	--

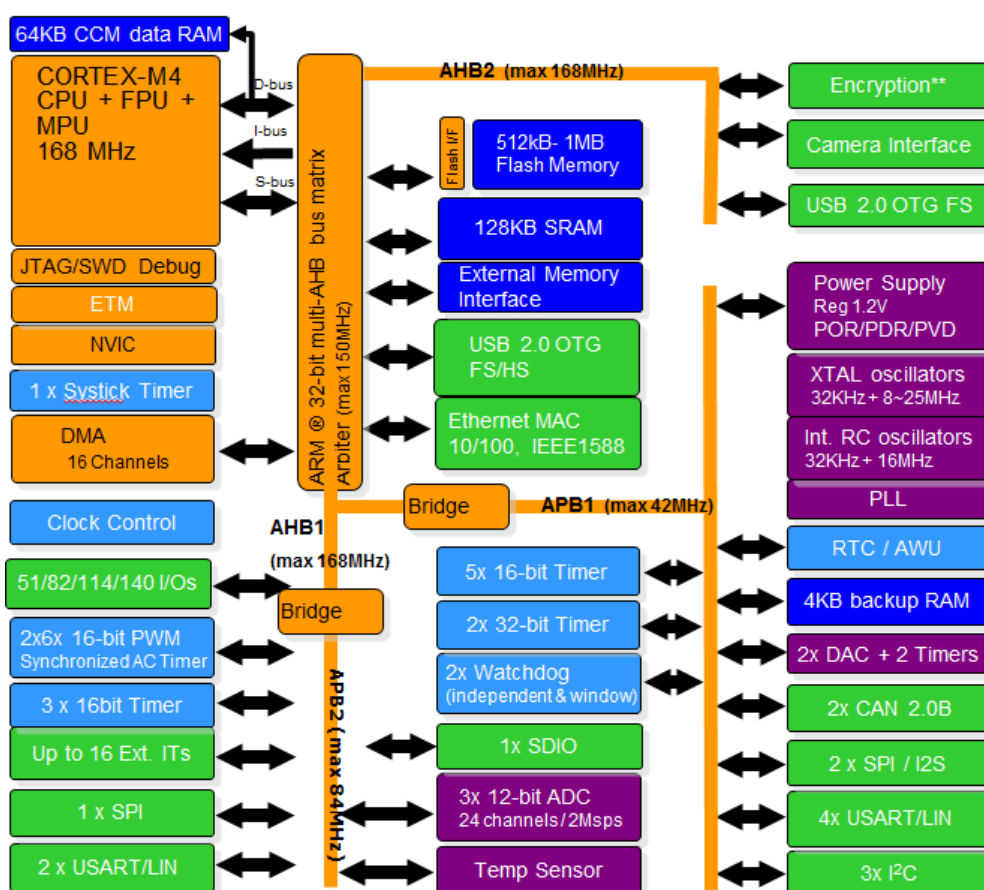
TARTALOMJEGYZÉK

2.20 Szoftverfejlesztés C nyelven Cortex M4 mikrokontrollerre	4
2.20.1 Programozás C nyelven mikrokontrollerre	7
2.20.2 Általános célú digitális ki/bemenetek kezelése - Mintapélda	11
2.20.3 Digitális I/O kezelése, megszakítás és időzítő használata - Mintapélda.....	23
2.20.4 Digitális I/O kezelése, időzítő használata PWM módban - Mintapélda.....	30
2.20.5 Az STM32 mikrokontroller inicializálása	32
2.20.6 SPI kommunikáció – Mintapélda	37
2.20.7 Analóg-digitális átalakító használata - Mintapélda	46

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 4. oldal
--	--	--

2.20 Szoftverfejlesztés C nyelven Cortex M4 mikrokontrollerre

A korábbi fejezetekben megismerkedhettünk egy viszonylag kis teljesítményű mikrokontroller architektúra – az Intel 8051 – assembly és C nyelvű szoftverfejlesztési lehetőségeivel, ezek ismeretében, nézzünk egy lényegesen komolyabb teljesítményű mikrokontrollert, mely napjainkra egyre szélesebb körben használt ARM magos mikrokontrollerek családjába tartozik, konkrétan a Cortex M4F családba. Az architektúrais alapok fejezetben volt már szó erről a processzorról, itt ismétlésképpen átnézzük programozási szempontból legfontosabb jellemzőit, és a legegyszerűbb perifériák kezelését.



1. ábra Az STM32F407VG mikrokontroller blokkvázlata

Az ábrán láthatjuk, hogy a mikrokontrollerben rengeteg periféria található – a kifejezetten egyszerű perifériáktól a bonyolult, nagysebességű perifériáig. Mi ezen jegyzet keretein belül csak néhány periféria típus kezelésébe próbálunk betekintést nyújtani, a többi periféria kezeléséhez a mikrokontroller leírására utalunk.

Láthatjuk, hogy rendelkezésünkre áll akár 140 általános célú ki/bemeneti port, használhatunk számos 16 és 32 bites – különböző extra funkciókkal is rendelkező – számláló/időzítő egységet valamint számos kommunikációs perifériát. Már az egyszerű

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontrollerek alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 5. oldal
---	---	---

perifériák kezeléséhez is elengedhetetlen a megszakítás vezérlő alrendszer, melynek működésébe szintén betekintést nyújtunk.

A Cortex M4 család programozását egyszerű mintapéldák megoldásával tekintjük át annak érdekében, hogy viszonylag gyorsan egyszerű lépésekben eljussunk működő, kipróbálható alkalmazásokhoz.

A mintapéldák mindegyikét az ST Microelectronics cég egyik legegyszerűbb olcsó fejlesztőkészletén az STM32F4 Discovery kártyára fogjuk elkészíteni. A fejlesztőkészlet ára a jegyzet készítésének időpontjában körülbelül 7 ezer forint, tehát mindenki számára elérhető gyakorlás céljára.



2. ábra Az STM32F4 Discovery fejlesztőkártya

A fejlesztőkártyán megtalálható minden olyan periféria, ami a Cortex M4 alapú szoftverfejlesztés alapvető fogásainak elsajátításához szükséges. A teljesség igénye nélkül a fejlesztőkártyán megtalálhatóak az alábbi eszközök:

- STM32F4 sorozatú mikrokontrollerek
- 4 különböző színű LED
- Nyomógomb
- MEMS gyorsulásérzékelő
- Audio egység
- USB OTG (On-The-Go) port
- SWD (Serial Wire Debug) debugger
- stb.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 6. oldal
--	--	--

A felsorolásból láthatjuk, hogy a fejlesztőkártyán rendelkezésre állnak egyszerűbb és bonyolultabb perifériák is annak érdekében, hogy a mikrokontroller képességeit kipróbálhassuk. Fontos megjegyezni, hogy a kártyán megtalálható a programozó/debugger áramkör is, amit akár későbbi STM32 alapú rendszereink felprogramozásához is használhatunk.

A mikrokontrollerben található perifériák több nagysebességű buszon keresztül kapcsolódnak a Cortex M4F maghoz. Programozási szempontból ennek azért van jelentősége, mert egyrészt minden egyes perifériának majd külön engedélyeznünk kell az órajelét – energiatakarékosági megfontolások miatt alapértelmezett módon minden periféria órajele le van kapcsolva. A másik ok, ami miatt szükségünk lehet már a legegyszerűbb feladat esetében is a buszrendszer ismeretére, mivel például egy egyszerű időzítési feladat megvalósításához is tudnunk kell azonosítani, hogy az adott időzítő melyik buszra csatlakozik, és az adott busz milyen sebességgel működik – ebből számíthatók az időzítők kívánt működéshez szükséges paraméterei.

A szoftverfejlesztéshez szükségünk van – a fejlesztőkártyán kívül – valamilyen szoftverfejlesztő környezetre is. Szoftverfejlesztő környezetek tekintetében számos választási lehetőségünk van, viszonylag széles a kínálat a Cortex magos processzorokhoz készült fejlesztőeszközök piacán. A legelterjedtebb fejlesztőeszközök képességei nagyon hasonlóak, mindenki ízlése, az eddigi megszokásai, illetve a pénztárcája alapján választhat fejlesztőeszközt.

Az ingyenesen elérhető, szabad forráskódú fejlesztőeszközök közé tartozik az Eclipse, mely a fejlesztői környezet felhasználói felülete, és ehhez az Eclipse rendszerhez kell keresni olyan cross-platform fordító eszközöket, melyek képesek együttműködni vele. Az Eclipse rendszerhez általában gcc alapú fordítót szokás illeszteni, ezen belül is olyat, amely képes Cortex processzoron futóképes kódot előállítani. Számos ilyen eszköz érhető el pl. CodeSourcery G++, Yagarto, stb.

Az ingyenesen elérhető fejlesztőkörnyezetek szinte kivétel nélkül az Eclipse-en alapulnak és gcc fordítót használnak.

Az ST Microelectronics az STM32 sorozatú mikrokontrollereihez az STM32CubeIDE fejlesztőeszközt ajánlja, mely ingyenesen elérhető a gyáró honlapján keresztül. Ez a fejlesztőeszköz jelenleg a cég saját termékének számít – korábban az Atollic nevű cég kezdte el a fejlesztését, ezt a céget az ST Microelectronics felvásárolta. Az STM32CubeIDE a felhasználói felülete Eclipse alapú, kiegészítve azt a fordítási folyamat részletes beállítására szolgáló modulokkal. Az STM32CubeIDE használata azért kifejezetten előnyös, mert képes komplett alkalmazásvázakat is generálni illeszkedve a kiválasztott konkrét mikrokontroller típushoz, illetve integráltan tartalmazza a mikrokontroller perifériáinak konfigurációját segítő kódgeneráló komponenseket. Az STM32CubeIDE fordítója is nyílt forráskódú eszközre épül, a gcc fordító egy módosított változatát használja.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 7. oldal
---	---	---

Az STM32 platformra történő szoftverfejlesztéshez készült – gyártói támogatással – az ingyenes Eclipse alapú System Workbench for STM32 (OpenSTM32) fejlesztőkörnyezet is. Mivel a telepítése kifejezetten egyszerű, és pontosan illeszkedik az STM32 platform sajátosságaihoz, így ez is kifejezetten jól használható fejlesztőeszköz az STM32 családra történő fejlesztéshez.

Ha a fizetős szoftverfejlesztő eszközök piacát nézzük, akkor a két legnagyobb fejlesztőeszköz gyártó a Keil és az IAR is kínál ARM magos processzorokhoz fejlesztőeszközöket. A Keil μ Vision, illetve az IAR Embedded Workbench széles körben használt, megbízható fejlesztőeszközök. Ezek az eszközök teljes egészében az adott cég termékei, tehát a felhasználói felület, és a fordítási eszközlánc is ugyanazon gyártótól származik.

A felsorolt fejlesztőeszközök mindegyike kereskedelmi termék, a teljes verziójukért komoly pénzeket kell fizetni természetesen, azonban mindegyiknek létezik olyan verziója, mely alkalmas kisebb feladatok megvalósítására, de ingyenesen elérhető, cserébe néhány kényelmi szolgáltatásról le kell mondanunk.

Az ismertetésre kerülő mintapéldákat az STM32CubeIDE környezetben készítettük, a csomagban megtalálható gcc fordítóval fordíthatóak.

Ahhoz, hogy elkezdhessük a fejlesztést a kiválasztott eszközünkre néhány dolgot érdemes előkészíteni:

- Szükségünk lesz a fejlesztőkártyára, valamint a fejlesztőeszközre
- Szükségünk lesz a fejlesztőkártya és a mikrokontroller dokumentációjára. A dokumentációk alapján tudjuk majd az egyes eszközök bekötését, elhelyezkedését kideríteni. A mikrokontroller dokumentációjából pedig a mikrokontroller belső felépítését, működését tudjuk megismerni. Az ARM magos mikrokontrollerek dokumentációja több részből áll. A mag dokumentációja szinte minden esetben az ARM honlapjáról érhető el, a konkrét mikrokontroller specifikus perifériák dokumentációja pedig a gyártótól szerezhető be.
- A fejlesztéshez a mikrokontroller és a fejlesztőpanel gyártója az ST Microelectronics – mint egyébként szinte minden gyártó – rendelkezésünkre bocsát egy periféria függvénykönyvtárat, melyben a perifériák alapvető kezeléséhez találunk előre megírt függvényeket.

2.20.1 Programozás C nyelven mikrokontrollerre

Mint azt a korábbi fejezetekben is említettük, az emberi gondolkodáshoz sokkal közelebb állnak a magasszintű nyelvek, ezért a programozók általában jobban kedvelik a magasszintű nyelveken történő alkalmazásfejlesztést, mint az alacsony szintű assembly kódban történő programozást. Szó volt már korábban arról is, hogy a mikrokontrollerek programozásához ideális választás lehet a C nyelv használata, mint szabványosított magasszintű programozási nyelv. A C nyelv választását sok indokkal alá lehet támasztani. C nyelven az algoritmikus problémák megoldása viszonylag egyszerűen elvégezhető, sokkal gyorsabban készíthetők működőképes alkalmazások mint bármilyen

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 8. oldal
--	--	--

alacsonyszintű nyelven. A C nyelv választása mellett szólhat még, hogy szinte minden processzor családhoz elérhető a piacon C fordító, valamint a nyelvet a programozók alapvetően ismerik. A C nyelvet egy általános célú programozási nyelvként hozták létre, ezért az olyan speciális feladatokat, melyeket egy mikrokontroller programozása során meg kell oldanunk, vagy a nyelv mikrokontroller specifikus kiegészítésével, vagy pedig a fordító eszköz megfelelő szolgáltatásainak használatával tudjuk megvalósítani.

A C nyelven történő mikrokontrolleres alkalmazás fejlesztése esetén az elvárásaink nem változnak, ugyanazt a feladatot meg kell tudni oldani, mint assembly nyelven. Elvárjuk ezen kívül, hogy a személyi számítógépek programozásánál megszokott általános, standard C utasítások, és a mikrokontrolleren értelmezhető funkciót végző függvénykönyvtárak használhatóak legyenek. Nem szeretnénk a memória feletti ellenőrzésről sem lemondani, hiszen attól, hogy magasszintű nyelven programozunk, a mikrokontrollerünk működése, és alapvető jellemzői nem változnak meg – a memória nem lesz több, a speciális rutinjainkat továbbra is speciális címekre kell helyeznünk, stb. A nyelvnek, illetve a fordítónak lehetővé kell tennie az assembly nyelven írt betétek elhelyezését a C nyelvű forráskódba, mivel előfordulhatnak olyan feladatok, melyeket esetleg csak assembly kóddal lehet megvalósítani, vagy pedig az assembly megvalósítás sokkal hatékonyabb.

A fordítóval szemben alapvető elvárás, hogy méret és sebesség szempontjából optimális kódot generáljon, tehát ne veszítsünk sokat se méret, se sebesség szempontjából az assembly nyelven történő programozáshoz képest. Az elterjedt, modern fordítók mindegyike fejlett optimalizációs képességekkel van felruházva. Számos felmérés és összehasonlítás alátámasztja, hogy a mai modern fordítók közel olyan méretű és sebességgel futó kódot képesek generálni, mint ha assemblyben írtuk volna meg azt.

A következőkben megismerkedünk a mikrokontrollerre történő C nyelvű alkalmazásfejlesztés alapjaival, kifejezetten az STM32 mikrokontroller családra koncentrálna.

Mint azt a korábbiakban tisztáztuk, a standard ANSI C önmagában nem alkalmas a mikrokontroller alapú rendszerek programozására. Ahhoz, hogy a mikrokontrolleren futtatható kód előállítható legyen a fordítónak különböző direktívákkal és vezérlőutasításokkal támogatnia kell azt, illetve néhány fordító esetében még a C nyelvhez is tesznek speciális kiegészítéseket, melyek természetesen illeszkednek a C nyelv alapvető filozófiájához és szintaktikájához. A standard C nyelv alapvetően PC-re történő alkalmazásfejlesztéshez lett kifejlesztve, és mint tudjuk a PC-k architektúrája Neumann modell szerinti, szemben a mikrokontrollerek Harvard, ill. módosított Harvard architektúrájával, ezzel a különbséggel is meg kell birkóznia a fordítóknak.

A C nyelv szintaktikája, nyelvi elemei szabványokban vannak meghatározva. A jegyzet írásának időpontjában három szabvány vonatkozik a C nyelvre, melyekben egyértelműen rögzítve van, hogy mit szabad és mit nem. Ezek a szabványok gyakorlatilag a C nyelv nyelvjárásainak tekinthetőek. Az újabb szabványok a korábbiak kiegészítései, a korábbi szabványokban nem egyértelműen specifikált elemeket specifikálják, illetve újabb nyelvi elemeket írnak le. A legrégebbi szabvány 1990-ben készült ez a C90, vagy más néven ANSI C. 1999-ben jelent meg a C99-es szabvány, majd a C11, 2011-ben, illetve a legújabb a C17, amely 2017-ben készült el és jelent meg.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 9. oldal
--	--	--

Az alkalmazásaink fordításakor a fordítónak meg kell adnunk, hogy mely szabvány szerint értelmezze az általunk megírt forráskódot.

Amikor egy fordítórendszert elkezdünk használni, érdemes megvizsgálni, hogy az egyes – C nyelvben használatos – változótípusokat milyen módon értelmezi, hány biten tárolja. Ez azért kifejezetten fontos, illetve szükséges, mert a C nyelvi szabványok nem minden változótípusnál definiálják fix méretüként az adott típust, csak a minimum bitszélességet határozzák meg (int, long, long long). Az alábbi táblázatban láthatjuk, hogy a gcc fordító milyen bitszélességen tárolja az egyes változótípusokat.

Adattípus	Előjeles	Bit	Bájt	Bájt (gcc)
signed char	Igen	8	1	1
unsigned char	Nem	8	1	1
signed short	Igen	16	2	2
unsigned short	Nem	16	2	2
signed int	Igen	>=16	>=2	4
unsigned int	Nem	>=16	>=2	4
signed long	Igen	>=32	>=4	4
unsigned long	Nem	>=32	>=4	4
signed long long	Igen	>=64	>=8	8
unsigned long long	Nem	>=64	>=8	8
float	Igen	32	4	4
double	Igen	64	8	8

1. táblázat Változótípusok bitszélessége

A változók használatakor vegyük tehát figyelembe a bitszélességet, és mindig olyat válasszunk, ami legjobban illeszkedik az adott mikrokontroller natív bitszélességéhez, és a tárolni kívánt értékek még elférnek benne.

A standard C nyelv részei a *const* és a *volatile* típusmódosítók, tehát nem mikrokontroller specifikusak, azonban érdemes megvizsgálnunk viselkedésüket.

Standard C-ben a *const* módosító használjuk konstansok létrehozására. A mikrokontrollerre történő fejlesztés során is megtehetjük ezt, azonban tartsuk szem előtt, hogy azok a változók, amelyeket *const* módosítóval deklarálunk csak olvashatóak lesznek, azonban a fordítótól függ, hogy a RAM-ban, vagy a ROM-ban kerülnek létrehozásra.

```
const int table[2][2] = { 0, 2, 4, 8 };
```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 10. oldal
---	---	--

A PC-re történő alkalmazásfejlesztés során viszonylag ritkán használjuk a *volatile* típusmódosítót, azonban mikrokontrollerre történő fejlesztés során kifejezetten fontos tudnunk, hogy mire jó, és milyen esetekben van rá szükségünk.

A *volatile* típusmódosító arra használható, hogy tudassuk a compilerrel: a változó értéke külső hatásra (nem csak a compiler által számításba vehető szekvenciális kódrészletek, hanem pl. a számára előre nem kiszámítható megszakítási rutin által) is változhat – ne optimalizálja a változóhoz történő hozzáféréseket, ne tartsa átmenetileg se regiszterben a változót a műveletek gyorsítása érdekében.

Nézzük az alábbi kódrészletet:

```
unsigned char reg1;    // Hardware Register #1
unsigned char reg2;    // Hardware Register #2

void func (void)
{
    while (reg1 & 0x01)    // Repeat while bit 0 is set
    {
        reg2 = 0x00;        // Toggle bit 7
        reg2 = 0x80;
    }
}
```

A reg1 változónk értékét nem programból állítjuk, hanem tételezzük fel, hogy ezt a mikrokontroller valamely perifériája állítja. A reg2 változót pedig arra szeretnénk használni, hogy amikor az értékét beállítjuk, akkor a mikrokontroller egy adott perifériájának állapota megváltozzék.

Első ránézésre úgy tűnik, hogy ez a pár soros függvény tökéletesen fog működni, azonban ha egy picit belegondolunk, hogy az optimalizáló compiler hogyan fordítaná le a függvényünket rájöhetünk, hogy alapvető működésbeli problémáink lesznek. Az egyik legegyszerűbb optimalizálási módszer, hogy azokat a kifejezéseket, melyek fordítási időben kiértékelhetők, ki is értékeli a compiler. A compiler tehát megvizsgálja az egyszerű függvényünket, és megállapítja, hogy a reg1 változónk értéke sehol sem változik, ezért előfordulhat hogy a while ciklusunk ciklusfeltételét még fordítási időben kiértékeli, ami azt eredményezheti, hogy a while ciklusunk egyszerűen kimarad a végleges kódból. Mi nem ezt várnánk, hanem azt, hogy minden egyes alkalommal a ciklusfeltétel kiértékelésre kerül, és a cikusból a feltételtől függően vagy kilépünk, vagy pedig lefut a ciklusmag.

A másik egyszerű optimalizálási lehetőség, melyet a legtöbb compiler alkalmaz, hogy az értelmetlen utasításokat elhagyja. Az egyszerű példánkban a reg2 változó használatával pedig az lehet a baj, hogy a compiler egyszerűen azt mondhatja, hogy két egymás utáni sorban állítjuk be ugyanazon változó értékét, a második úgyis felülírja az elsőt, tehát az első értékadást felesleges. Ez a működés nem elfogadható számunkra, mert lehet hogy a periféria, melyet a reg2 változón keresztül elérni próbálunk pontosan a két egymás utáni értékadásra végzi el a kívánt funkciót.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 11. oldal
--	--	---

Ahhoz, hogy az ilyen lehetséges optimalizálásokat elkerüljük, minden hasonló esetben a változóinkat el kell látnunk a *volatile* típusmódosítóval, ahogy ezt az alábbi példában is láthatjuk.

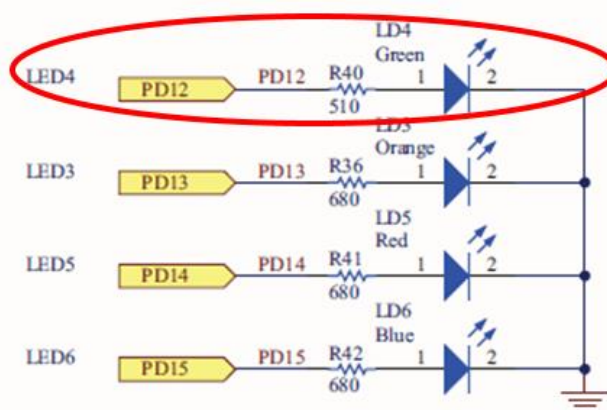
```
volatile unsigned char reg1;    // Hardware Register #1
volatile unsigned char reg2;    // Hardware Register #2

void func (void)
{
    while (reg1 & 0x01)          // Repeat while bit 0 is set
    {
        reg2 = 0x00;             // Toggle bit 7
        reg2 = 0x80;
    }
}
```

2.20.2 Általános célú digitális ki/bemenetek kezelése - Mintapéllda

A fejlesztőkörnyezet és a szükséges dokumentációk birtokában hozzákezdhetünk a fejlesztéshez. Első körben egy kifejezetten egyszerű feladat megoldását tűzzük ki célul. Ezt a feladatot szoktuk általában egy ismeretlen mikrokontrolleren először elkészíteni. A feladat, mint említettem nem túl bonyolult: villogtassuk az STM32F4Discovery fejlesztőkártyán található zöld színű LED-et szoftveres várakozást implementálva. A várakozással szemben csak annyi követelményt tűzünk ki magunk elé, hogy a villogás szemmel látható legyen.

Első lépésben elő kell vennünk a fejlesztőkártya dokumentációját, és meg kell keresnünk benne az eszköz kapcsolási rajzát. A kapcsolási rajzot tanulmányozzuk, és megkeressük benne a LED-eket tartalmazó részt. Az alábbi 3. ábra pirossal bekarikázott részén láthatjuk a zöld LED-et. A kapcsolási rajz alapján két fontos információt szerezhetünk meg, egyrészt láthatjuk, hogy a LED a mikrokontroller PD12 lábához van csatlakoztatva – ez a D port 12-es lába – illetve azt is láthatjuk, hogy a LED vezérlése egyenes logika szerint történik, tehát a LED akkor fog világítani, amikor a port-lábat logikai 1 szintre állítjuk.



3. ábra Az STM32F4 Discovery kapcsolási rajzának LED illesztést leíró része

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 12. oldal
---	---	--

Most, hogy azonosítottuk, hogy melyik portra csatlakozik a vezérelni kívánt LED, a mikrokontroller dokumentációja segítségével derítsük ki, hogyan lehet vezérelni azt. A dokumentációból több fontos dolgot is kideríthetünk:

- A mikrokontroller belső perifériái fogyasztáscsökkentési megfontolások miatt nem kapnak reset után automatikusan órajelet, ezt külön engedélyezni szükséges.
- A GPIO (általános célú ki/bemeneti) lábak 16 bites portokba vannak szervezve, ill. a GPIO lábak alapértelmezés szerint bemenetek (ez minden mikrokontrollerre igaz kicsitől a nagyig). Amennyiben a mikrokontroller lábai alapértelmezés szerint kimenetek lennének, a kontroller feléledésekor mindaddig összehajtanának a bemenetként használni kívánt lábak és a külső perifériák, amíg a lábakat befelé nem fordítanánk. A kimenetként használni kívánt lábakra csatlakozó áramköröket fel- illetve lehúzó ellenállásokkal szokás definit állapotba hozni.

A dokumentációk áttanulmányozása után az alapvető információk már rendelkezésünkre állnak, tervezzük meg az alkalmazásunkat, gondoljuk végig, hogy milyen lépéseket kell elvégeznünk egy a célnak megfelelő alkalmazás elkészítéséhez:

1. Periféria órajelének engedélyezése – D GPIO port
2. Port inicializálás
 1. Port sebesség beállítása
 2. Port irány beállítása – kimenet legyen
 3. Kimenet típusának beállítása – push-pull mód
 4. Felhúzó ellenállások beállítása – letiltva
3. LED-ek ki/be kapcsolása, közben várakozás

A feladatunk megvalósításához két út közül választhatunk: az egyik, hogy még részletesebben áttanulmányozzuk a dokumentációt, megnézzük, hogy mely regisztereket kell a kívánt eredmény eléréséhez módosítanunk, beállítanunk, míg a másik irány, hogy használjuk a gyártó által biztosított periféria függvénykönyvtárat.

A periféria függvénykönyvtár, mint az korábban már említettük segítséget nyújt a mikrokontrollerben található legtöbb periféria alapszintű kezeléséhez. Nagy előnye, hogy nem kell pontosan ismernünk a mikrokontroller regiszterek neveit és bitkiosztását, hanem periféria csoportonként beszédes nevű függvényeket és konstansokat használva érhetjük el ugyanazt az eredményt, mint ha közvetlenül a regisztereket állítanánk. Csodára természetesen ne számítsunk – a periféria függvénykönyvtár is a mikrokontroller regisztereit állítja, használatának előnye abban mutatkozhat meg, hogy sokkal gyorsabban lehet vele fejleszteni, mint ha közvetlenül a regisztereket állítanánk.

A függvénykönyvtár számos c és h fájlból áll, melyek elnevezése utal a bennük kezelt periféria csoportra. Az alábbi táblázatban láthatjuk, hogy mely fájlok alkotják a periféria függvénykönyvtárat – a teljesség igénye nélkül.

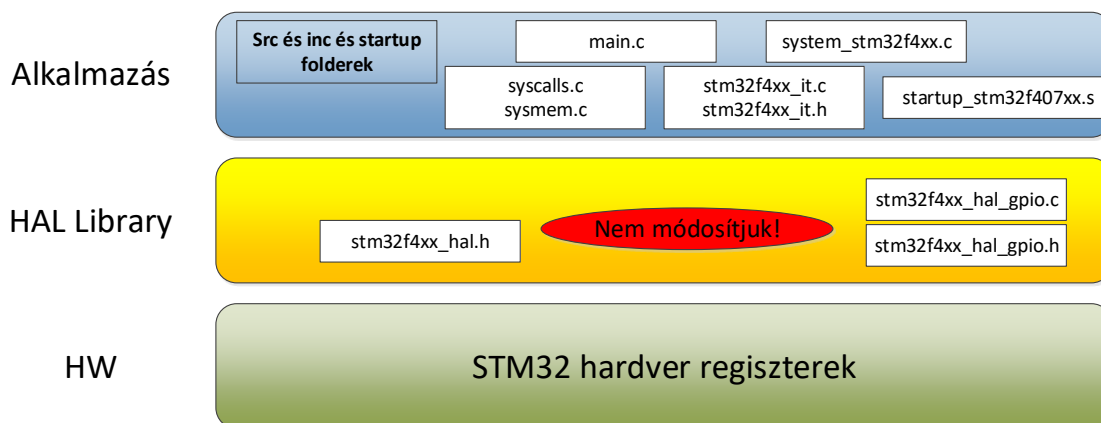
BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 13. oldal
--	--	---

C és H fájl neve	Perifériacsoport, funkció	C és H fájl neve	Perifériacsoport, funkció
stm32f4xx_hal_adc	Analóg-digitális átalakító	stm32f4xx_hal_i2c	I2C interfész
stm32f4xx_hal_can	CAN interfész	stm32f4xx_hal_iwdg	Watchdog
stm32f4xx_hal_crc	CRC alrendszer	stm32f4xx_hal_pwr	Tápellátás
stm32f4xx_hal_cryp	Titkosítás	stm32f4xx_hal_rcc	Reset és órajel beállítások
stm32f4xx_hal_dac	Digitális-analóg átalakító	stm32f4xx_hal_rng	Véletlenszám generátor
stm32f4xx_hal_dcmi	Digitális kamera interfész	stm32f4xx_hal_rtc	Valós idejű óra
stm32f4xx_hal_dma	DMA	stm32f4xx_hal_sd	SD kártya interfész
stm32f4xx_hal_flash	Flash támogatás	stm32f4xx_hal_spi	SPI interfész
stm32f4xx_ll_fsmc	Statikus memória vezérlő	stm32f4xx_hal_tim	Időzítők
stm32f4xx_hal_gpio	Általános Ki/bemenetek	stm32f4xx_hal_usart	USART interfész
stm32f4xx_hal_hash	Hash támogatás	stm32f4xx_hal_wwdg	Watchdog

4. ábra Az ST periféria függvénykönyvtár fájllai

A periféria függvénykönyvtár megkönnyíti a fejlesztést, azonban a mikrokontroller működésével ekkor is tisztában kell lennünk – csak a regiszterek konkrét elnevezését nem szükséges ismernünk. A függvénykönyvtárban megtalálható forráskódokból ötleteket is lehet meríteni, az egyes perifériák kezeléséhez, ez akkor lehet hasznos, ha olyan forráskódot kell készítenünk, amelyben nem használhatunk külső könyvtárakat, ebben az esetben nekünk kell megírni a periféria kezelő rutinokat a függvénykönyvtár implementációja alapján.

Kezdjük el fejleszteni az alkalmazásunkat az STM32CubeIDE segítségével. A fejlesztőkörnyezetünk tartalmaz egy projekt varázslót, melynek segítségével az STM32F4 sorozatú mikrokontrollerekre tudunk projektet generálni. A varázsló előállítja nekünk az alkalmazásunk vázát, mely az alábbiak szerint fog felépülni.



5. ábra Az STM32 HAL könyvtár alapú alkalmazásunk felépítése

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 14. oldal
--	--	---

A main.c fájlban lesz megtalálható az alkalmazásunk main függvénye. A mikrokontroller inicializálásához szükségesek a startup_stm32f407xx.s és a system_stm32f4xx.c fájlok, melyben a mikrokontroller órajeleit, az adatmemória tartalmát, és a stack-et illetve a periféria függvénykönyvtárat inicializáljuk, valamint beregisztráljuk az alapértelmezett megszakítás kezelő rutinokat. Az stm32f4xx_hal_conf.h fájl a periféria függvénykönyvtár konfigurációját tartalmazza (ebben kell becsatolnunk azokat a könyvtári definíciós fájlokat, amiket használni szeretnénk). A projektünkben megtalálható lesz ezeken kívül a függvénykönyvtár összes fájlja – ezeket ne módosítsuk!

Az alkalmazásunk tervében első lépésként a GPIOD periféria órajelének engedélyezését tűztük ki célul, keressünk ehhez a feladathoz a függvénykönyvtárban egy alkalmas függvényt. Az órajelek kezeléséhez szükséges feladatokat az RCC (Reset and Clock Configuration) modul végzi a mikrokontrolleren belül, az ehhez a modulhoz tartozó könyvtári fájlok az stm32f4xx_hal_rcc.c és h. Megnyitva ezen fájlokat végignézhetjük, hogy milyen függvényeket és makrókat tartalmaznak – minden függvény mellett találunk egy leírást is az általa végzett funkcióról és a paraméterezésről.

Keressünk olyan függvényt vagy makrót a könyvtárban, mely GPIOD periféria órajelét engedélyezi:

```
_GPIOD_CLK_ENABLE();
```

Második lépésként a GPIOD port inicializálását kellene elvégeznünk, keressünk hozzá egy függvényt az stm32f4xx_hal_gpio fájlokban.

```
void HAL_GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_Init);
```

Milyen paraméterekkel kellene meghívunk a függvényt? A függvény első paramétere egy könyvtári konstans - a port memóriacíme, a második paraméter pedig egy struktúra, melyet kitöltve megadhatjuk, hogy miként viselkedjen az adott port. A kitöltendő struktúra definíciója az alábbiakban látható.

```
typedef struct
{
    uint32_t Pin;           /* GPIO Pin */
    uint32_t Mode;          /* Mode */
    uint32_t Speed;         /* Speed */
    uint32_t Alternate;     /* Alternate function */
    uint32_t Pull;          /* Pull-up/Pull down */
}GPIO_InitTypeDef;
```

A struktúra kitöltésével meg tudjuk mondani az egyes port lábakhoz, hogy milyen módban működjenek (ki/bemenet), milyen kimeneti meghajtást használjanak (Open-drain/Push-Pull), legyenek-e bekapcsolva a felhúzó ellenállások, milyen sebességgel működjön a port, illetve szeretnénk-e a kiválasztott lábakra valamely más belső periféria jelét kivezetni.

A struktúra tagjait beszédes nevű előre definiált konstansok segítségével tölthetjük ki pl.: GPIO_PIN_12, GPIO_MODE_OUT_PP, stb.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 15. oldal
--	--	---

Az inicializálás után a tényleges feladatunkat kellene elvégeznünk, tehát LED-et kellene ki, illetve bekapcsolni. Amint azt korábban már láttuk, a LED a D port 12-es lábára csatlakozik, tehát a D port 12-es lábának értékét kellene befolyásolnunk a LED megfelelő működéséhez.

A függvénykönyvtárban keresgélve az alábbi függvényeket találhatjuk kimeneti port értékbeállításához.

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef* GPIOx, uint16_t
GPIO_Pin);
void HAL_GPIO_WritePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin,
GPIO_PinState PinState);
void HAL_GPIO_TogglePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
```

Láthatjuk, hogy több lehetőségünk is van a port lábak értékének beállítására: van függvényünk, amely a megadott lábakat állítja a kívánt logikai 0 vagy 1 szintre, illetve van olyan függvényünk, mely a megadott lábak állapotát negálja.

A mi céljainkra a HAL_GPIO_TogglePin függvény lesz a megfelelő, amely a paraméterek által meghatározott lábak állapotát negálja.

Kiválasztottunk tehát minden használni kívánt függvényt, írjuk meg az alkalmazásunkat.

```
#include "stm32f4xx_hal.h"
int main(int argc, char* argv[])
{
    int i;
    GPIO_InitTypeDef GPIO_InitStruct;
    /* GPIOD periféria órajel engedélyezés */
    __GPIOD_CLK_ENABLE();

    /* GPIO láb konfigurálása */
    GPIO_InitStruct.Pin = GPIO_PIN_12; /* 12-es láb */
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP; /* kimenet p/p mód */
    GPIO_InitStruct.Pull = GPIO_NOPULL; /* nincs felhúzó ell. */
    GPIO_InitStruct.Speed = GPIO_SPEED_LOW; /* kis sebesség */
    HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
    while (1)
    {
        /* Invertáljuk a LED állapotát */
        HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_14);
        /* Várakozunk egy for ciklussal */
        for(i=0;i<0x3ffffff;i++);
    }
}
```

Az elkészített alkalmazásban létrehozunk két lokális változót `i` és `GPIO_InitStruct` néven. Az `i` változónkat a szoftveres várakozáshoz, a `GPIO_InitStruct` változónkat pedig a port beállításához fogjuk használni. Következő lépésben engedélyezzük a D port órajelét, majd pedig kitöltjük a port inicializálásához a `GPIO_InitStruct` struktúrát. Beállítjuk, hogy melyik lábakra legyen érvényes a beállításunk a struktúra első tagjával, majd pedig megmondjuk, hogy kimeneti módban szeretnénk használni a lábat. Ezen kívül a meghajtás típusát mindkét irányban aktívra (push-pull) állítjuk, a port sebességnek

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 16. oldal
--	--	---

2MHz-et választunk, illetve a felhúzó ellenállást kikapcsoljuk. A beállításaink érvényre juttatásához a HAL_GPIO_Init függvényt meghívjuk a megfelelő port és inicializáló struktúra megadásával.

Az inicializáló rész után következik a tényleges funkciót megvalósító végtelen ciklusba szervezett főhurok. A portláb értékét invertáljuk a HAL_GPIO_TogglePin függvénnyel – paramétereiben megadjuk a portot és az invertálni kívánt lábakat. Majd várakozunk egy for ciklussal – ami nem kifejezetten elegáns, általában nem szoktunk szoftveres hurokkal várakozni, mivel ilyenkor a mikrokontroller műveletvégző egységét dolgoztatjuk olyan feladattal, amit egy sokkal egyszerűbb céleszközzel is meg lehetne oldani – ezért általában várakozáshoz, időzítéshez minden mikrokontroller tartalmaz időzítő perifériákat – ezekre a későbbiekben még részletesebben kitérünk.

Láthatjuk, hogy az alkalmazásunk nem túl hosszú, viszonylag könnyen megérthető lépésekből áll. Nézzük meg, hogy mit spóroltunk azzal, hogy nem magunk állítgattuk a regiszterek tartalmát, hanem függvénykönyvtárat használtunk. Az órajel engedélyező makrónk kifejezetten egyszerű, láthatjuk hogy egy regiszter megfelelő bitjét állítjuk be, ezzel nem spóroltunk sokat.

```
#define __GPIO_CLK_ENABLE() (RCC->AHB1ENR |= (RCC_AHB1ENR_GPIODEN))
```

Jegyezzük meg, hogy ha egy szám tetszőleges bitjeibe 1 értéket szeretnénk írni, akkor mindig bitenkénti VAGY ($a = a | b$, tömörebb írásmóddal $a |= b$) operátort kell használnunk. A módosítani kívánt számunk „vagy” kapcsolatát kell, képezzük egy olyan számmal melyben a beállítani kívánt bitek 1 értékűek, a többi bit pedig nulla. Ha egy szám tetszőleges bitjeibe nullát szeretnénk beírni, akkor mindig bitenkénti ÉS operátort ($a = a \& b$, tömörebb írásmóddal $a \&= b$) kell használnunk. A módosítani kívánt számunk „és” kapcsolatát kell, képezzük egy olyan számmal, melyben a módosítani kívánt bitek 0 értékűek, a többi pedig 1.

Nézzük ezek után a port működési módját beállító HAL_GPIO_Init függvényünket.

```
void HAL_GPIO_Init(GPIO_TypeDef *GPIOx, GPIO_InitTypeDef *GPIO_Init)
{
    uint32_t position;
    uint32_t ioposition = 0x00;
    uint32_t iocurrent = 0x00;
    uint32_t temp = 0x00;

    /* Check the parameters */
    assert_param(IS_GPIO_PIN(GPIO_Init->Pin));
    assert_param(IS_GPIO_MODE(GPIO_Init->Mode));
    assert_param(IS_GPIO_PULL(GPIO_Init->Pull));

    /* Configure the port pins */
    for(position = 0; position < GPIO_NUMBER; position++)
    {
        /* Get the IO position */
        ioposition = ((uint32_t)0x01) << position;
```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 17. oldal
---	---	--

```

/* Get the current IO position */
iocurrent = (uint32_t)(GPIO_Init->Pin) & ioposition;

if(iocurrent == ioposition)
{
    /*----- GPIO Mode Configuration -----*/
    /* In case of Alternate function mode selection */
    if((GPIO_Init->Mode == GPIO_MODE_AF_PP) ||
        (GPIO_Init->Mode == GPIO_MODE_AF_OD))
    {
        /* Check the Alternate function parameter */
        assert_param(IS_GPIO_AF(GPIO_Init->Alternate));
        /* Configure Alternate function mapped with the current IO */
        temp = GPIOx->AFR[position >> 3];
        temp &= ~((uint32_t)0xF <<
            ((uint32_t)(position & (uint32_t)0x07) * 4)) ;
        temp |= ((uint32_t)(GPIO_Init->Alternate) <<
            (((uint32_t)position & (uint32_t)0x07) * 4));
        GPIOx->AFR[position >> 3] = temp;
    }

    /* Configure IO Direction mode (Input, Output, Alternate or
        Analog) */
    temp = GPIOx->MODER;
    temp &= ~(GPIO_MODER_MODER0 << (position * 2));
    temp |= ((GPIO_Init->Mode & GPIO_MODE) << (position * 2));
    GPIOx->MODER = temp;

    /* In case of Output or Alternate function mode selection */
    if((GPIO_Init->Mode == GPIO_MODE_OUTPUT_PP) ||
        (GPIO_Init->Mode == GPIO_MODE_AF_PP) ||
        (GPIO_Init->Mode == GPIO_MODE_OUTPUT_OD) ||
        (GPIO_Init->Mode == GPIO_MODE_AF_OD))
    {
        /* Check the Speed parameter */
        assert_param(IS_GPIO_SPEED(GPIO_Init->Speed));
        /* Configure the IO Speed */
        temp = GPIOx->OSPEEDR;
        temp &= ~(GPIO_OSPEEDER_OSPEEDR0 << (position * 2));
        temp |= (GPIO_Init->Speed << (position * 2));
        GPIOx->OSPEEDR = temp;

        /* Configure the IO Output Type */
        temp = GPIOx->OTYPER;
        temp &= ~(GPIO_OTYPER_OT_0 << position) ;
        temp |= (((GPIO_Init->Mode & GPIO_OUTPUT_TYPE) >> 4) <<
            position);
        GPIOx->OTYPER = temp;
    }

    /* Activate the Pull-up/down resistor for the current IO */
    temp = GPIOx->PUPDR;
    temp &= ~(GPIO_PUPDR_PUPDR0 << (position * 2));
    temp |= ((GPIO_Init->Pull) << (position * 2));
    GPIOx->PUPDR = temp;
}

```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 18. oldal
--	--	---

```

/*----- EXTI Mode Configuration -----*/
/* Configure the External IT or event for the current IO */
if((GPIO_Init->Mode & EXTI_MODE) == EXTI_MODE)
{
    /* Enable SYSCFG Clock */
    __SYSCFG_CLK_ENABLE();

    temp = SYSCFG->EXTICR[position >> 2];
    temp &= ~(((uint32_t)0x0F) << (4 * (position & 0x03)));
    temp |= ((uint32_t)(GET_GPIO_SOURCE(GPIOx)) <<
              (4 * (position & 0x03)));
    SYSCFG->EXTICR[position >> 2] = temp;

    /* Clear EXTI line configuration */
    temp = EXTI->IMR;
    temp &= ~((uint32_t)iocurrent);
    if((GPIO_Init->Mode & GPIO_MODE_IT) == GPIO_MODE_IT)
    {
        temp |= iocurrent;
    }
    EXTI->IMR = temp;

    temp = EXTI->EMR;
    temp &= ~((uint32_t)iocurrent);
    if((GPIO_Init->Mode & GPIO_MODE_EVT) == GPIO_MODE_EVT)
    {
        temp |= iocurrent;
    }
    EXTI->EMR = temp;

    /* Clear Rising Falling edge configuration */
    temp = EXTI->RTSR;
    temp &= ~((uint32_t)iocurrent);
    if((GPIO_Init->Mode & RISING_EDGE) == RISING_EDGE)
    {
        temp |= iocurrent;
    }
    EXTI->RTSR = temp;

    temp = EXTI->FTSR;
    temp &= ~((uint32_t)iocurrent);
    if((GPIO_Init->Mode & FALLING_EDGE) == FALLING_EDGE)
    {
        temp |= iocurrent;
    }
    EXTI->FTSR = temp;
}
}
}
}

```

Láthatjuk, hogy ebben a függvényben már több regiszter értékét kell állítanunk a kívánt eredmény elérésének érdekében, tehát viszonylag sokat spóroltunk a függvény használatával.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 19. oldal
---	---	--

Nézzük a harmadik könyvtári függvényünket, a HAL_GPIO_TogglePin függvényt.

```
void HAL_GPIO_TogglePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
{
    /* Check the parameters */
    assert_param(IS_GPIO_PIN(GPIO_Pin));

    GPIOx->ODR ^= GPIO_Pin;
}
```

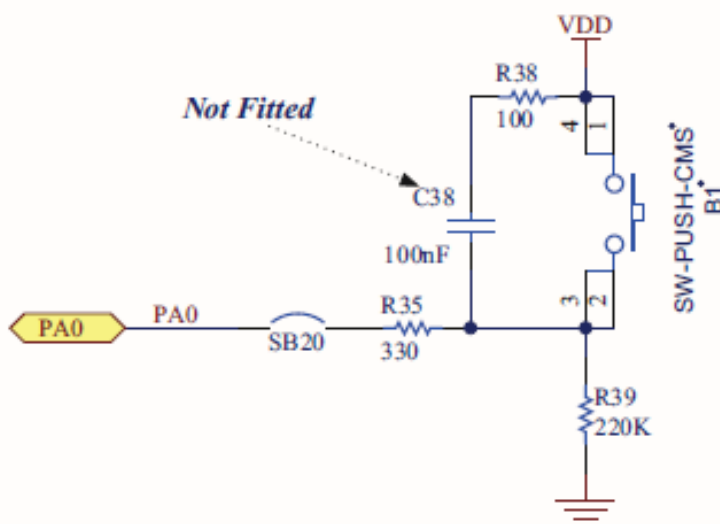
Ennél a függvénynél tényleg nem spóroltunk sokat, de ha belegondolunk, nem is igazán számíthattunk túl komoly feladatra, várható volt, hogy a kimeneti regiszter (ODR) megfelelő bitjeit kell módosítanunk.

A függvény implementációjában láthatjuk, hogy a bitek negálásához a kizáró vagy műveletet használtuk.

Jegyezzük meg, hogy ha egy szám tetszőleges bitjeit negálni szeretnénk, akkor mindig **KIZÁRÓ VAGY** ($a = a \oplus b$, illetve $a \oplus b$) operátort kell használnunk. A módosítani kívánt számunk kizáró vagy kapcsolatát kell, képezzük egy olyan számmal melyben a negálni kívánt bitek 1 értékűek, a többi bit pedig nulla – a logikai azonosságoknál megtanult módon $a \oplus 0 = a$, illetve $a \oplus 1 = \neg a$.

Az általános célú ki/bemeneti portok a mikrokontrollerek egyszerűbb perifériái közé tartoznak, túl sok beállítási lehetőségük sincs, de így is elég sok munkát spóroltunk azzal, hogy függvénykönyvtárat használtunk. A megtakarítás bonyolultabb nagysebességű perifériáknál – ahol esetenként 30-40 regisztert kell beállítani ahhoz, hogy a periféria megfelelő módon működjön – még számottevőbb.

Egészítsük ki a mintapéldánkat egy új funkcióval: a villogást lehessen megállítani és újraindítani a fejlesztőpanelen található nyomógomb megnyomásával.



6. ábra Az STM32F4 Discovery kapcsolási rajzának nyomógomb illesztést leíró része

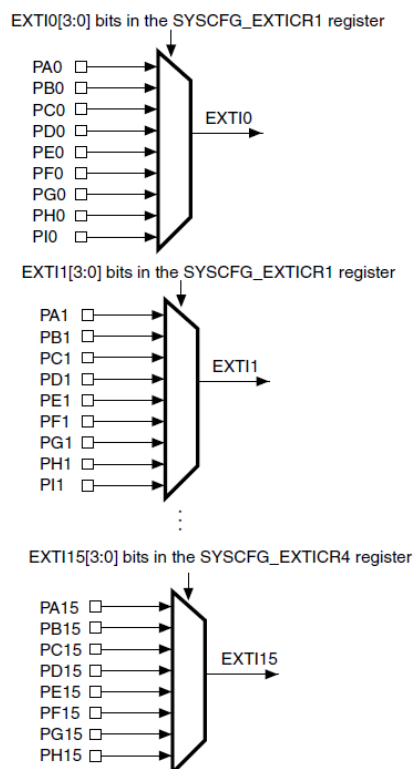
BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 20. oldal
--	--	---

A kártya kapcsolási rajzában a következő – számunkra ebben a feladatban lényeges – része a nyomógomb illesztését leíró rész. Láthatjuk, hogy a nyomógomb a mikrokontroller PA0 lábához kapcsolódik (A port, 0-s láb), és ha megnyomjuk a gombot, akkor a mikrokontroller lábat felhúzza tápfeszültségre – azaz logikai 1 szintre.

A nyomógombot megszakítással szeretnénk kezelni, ehhez meg kell ismerkedünk a mikrokontroller megszakításkezelő rendszerének működésével, konfigurációs lehetőségeivel.

A külső megszakításokkal a mikrokontroller dokumentációja külön fejezetben foglalkozik. Nézzük át a legfontosabb jellemzőit a mikrokontroller külső megszakításokat kezelő EXTI moduljának. A mikrokontroller minden GPIO lába lehet külső megszakítás forrása, természetesen konfigurálható, hogy mely lábak generáljanak ténylegesen megszakítást.

A mikrokontrollerben található megszakítás vezérlő modulnak (NVIC) 16 általános célú külső megszakítás vonala van. A 16 vonalra kapcsolódnak a GPIO portok vonalai, mégpedig úgy, hogy a 0-s megszakítás vonalra az összes port 0-s lába, az 1-esre az össze port 1-es lába és így tovább a tizenötödik külső megszakításvonalig.



7. ábra A külső megszakításvonalak kapcsolódása a GPIO lábakhoz

A megszakítás vezérlő feladata a megszakításkérések – prioritásuknak megfelelően – érvényre juttatása. Az STM32F4 mikrokontrollerben található megszakítás vezérlőhöz 87+16 megszakításvonal kapcsolódik, 87 megszakítás vonalat a perifériák használnak, a

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 21. oldal
--	--	---

maradék 16-ot pedig a Cortex M4 processzormag használja. A megszakítás vezérlőben beállítható, hogy milyen megszakítás prioritásmodell szerint működjön. A megfelelő regiszterekben 4 bit áll rendelkezésre a megszakítások prioritásának tárolására, ennek a négy bitnek az értelmezését szabja meg a kiválasztott prioritásmodell.

PRIGROUP (3 bits)	Binary Point (group.sub)		Preempting Priority (Group Priority)		Sub-Priority	
			Bits	Levels	Bits	Levels
011 (NVIC_PriorityGroup_4)	4.0	gggg	4	16	0	0
100 (NVIC_PriorityGroup_3)	3.1	gggs	3	8	1	2
101 (NVIC_PriorityGroup_2)	2.2	ggss	2	4	2	4
110 (NVIC_PriorityGroup_1)	1.3	gsss	1	2	3	8
111 (NVIC_PriorityGroup_0)	0.4	ssss	0	0	4	16

8. ábra Az NVIC prioritásmodellek értelmezése

Összesen ötféle prioritásmodell közül lehet választani, melyek azt határozzák meg, hogy a rendelkezésre álló négy bitből hány bitet használunk csoport prioritáshoz, és hányat csoporton belüli al-prioritáshoz. A megszakítás vezérlő mindig a magasabb csoport prioritású megszakításokat engedi érvényre jutni, ezek az alacsonyabb csoport prioritású megszakításokat is megszakíthatják. Az al-prioritás azt szabja meg, hogy az egyszerre bekövetkező azonos csoportú megszakításokat milyen sorrendben fogja a vezérlő érvényre juttatni. A megszakítás vezérlőben a kisebb számok jelentik a magasabb prioritásokat.

Tehát ahhoz, hogy a nyomógombunkat megszakításokkal tudjuk kezelni egyrészt a GPIO port megfelelő lábát bemenetként kell inicializálnunk és be kell állítanunk, hogy a megfelelő jelváltás hatására megszakítás generálódjon. Ezen kívül a megszakítás vezérlő alrendszer beállítása után a külső megszakítás prioritását be kell állítani, majd engedélyezni. Végül a megszakítást le kell kezelnünk.

Első lépésben tehát a GPIO periféria inicializálását kell elvégeznünk.

```

__GPIOA_CLK_ENABLE();

portInit.Mode = GPIO_MODE_IT_FALLING;
portInit.Speed = GPIO_SPEED_LOW;
portInit.Pull = GPIO_NOPULL;
portInit.Pin = GPIO_PIN_0;
HAL_GPIO_Init(GPIOA, &portInit);

```

Az inicializálás lépései nagyban hasonlítanak az előbbiekhöz: első lépésben engedélyezzük a GPIOA port órajelét, majd beállítjuk az inicializáló struktúra tagjait.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 22. oldal
--	--	---

Értelem szerint a beállításoknak a port 0-s lábára kell érvényesnek lennie, ezért a struktúra Pin tagja mindenképp a GPIO_PIN_0 értéket kell hogy kapja. A fel/lehúzó ellenállásokra továbbra sincs szükségünk, és az alacsony sebesség továbbra is elegendő, ezért a GPIO_NOPULL és a GPIO_SPEED_LOW beállításokat használjuk. A mód beállításnál azt kell beállítanunk, hogy a kiválasztott láb bemenetként funkcionáljon és lefutó élre megszakítás generálódjon, ezt a GPIO_MODE_IT_FALLING opcióval tudjuk megtenni. Végül a struktúra kitöltése után a HAL_GPIO_Init függvény meghívásával végezzük el a GPIO port inicializálását.

Következő lépésben a külső megszakítással kapcsolatos beállításokat kell elvégeznünk. Első lépésben kiválasztjuk a prioritásmodellt, tehát megmondjuk, hogy hány prioritás csoportunk legyen – mi olyan beállítást választunk, hogy két csoportunk legyen (jelen alkalmazásban nincs komoly jelentősége, mivel egyetlen megszakítást kezelünk).

```
/* NVIC Prioritás modell beállítása */
HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_1);

/* Megszakítás prioritásának beállítása */
HAL_NVIC_SetPriority(EXTI0_IRQn, 0, 0);

/* Megszakítás engedélyezése */
HAL_NVIC_EnableIRQ(EXTI0_IRQn);
```

Majd a HAL_NVIC_SetPriority függvény meghívásával beállítjuk a külső megszakításunk prioritását, illetve a HAL_NVIC_EnableIRQ függvénnyel engedélyezzük azt.

Az alkalmazás inicializációs részeivel ezzel el is készültünk, már „csak” a megszakítást kell lekezelnünk, illetve az alkalmazáslogikát módosítanunk a LED engedélyezéshez ill. tiltáshoz.

```
void EXTI0_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
}
```

A natív – használt periféria könyvtártól független – megszakítás kezelő függvényben meghívjuk a függvénykönyvtárunk beépített megszakítás kezelő függvényét. Ez azzal az előnnyel jár számunkra, hogy a megszakítás jelző flag állapotának vizsgálatát és a flag törlését a könyvtári függvény elvégzi helyettünk, majd pedig meghívja az adott megszakítás típusához tartozó callback függvényt. A callback függvény a legtöbb függvénykönyvtárban olyan függvényt jelent, melyet a fejlesztőnek kell megírnia, és a függvénykönyvtár megfelelő függvényei ezt a megírt callback függvényt hívják meg.

```
static uint8_t LedEnable;
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if ( GPIO_Pin & GPIO_PIN_0 )
        LedEnable = !LedEnable;
```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 23. oldal
--	--	---

```

}
int main(int argc, char* argv[])
{
    /* Inicializációs lépések */
    while (1)
    {
        /* Invertáljuk a LED állapotát */
        if (LedEnable)
            HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_12);
        /* Várakozás */
    }
}

```

Az alkalmazáslogikánk annyiban módosul, hogy létrehozunk egy globális LedEnable változót, melynek értéke azt fogja meghatározni, hogy villogjon-e a LED vagy sem – ezt láthatjuk a main függvényben a while ciklus magjában.

A külső megszakítás hatására meghívódó callback függvényben invertáljuk a LedEnable változónk állapotát. A callback függvényben kötelezően vizsgálni kell, hogy melyik vonalon érkezett a megszakítás – mivel minden külső megszakítás esetén ugyanez a callback függvény hívódik meg. Tehát a LedEnable változót csak akkor szabad negálnunk, ha a megszakításunk a 0-s vonalon érkezett.

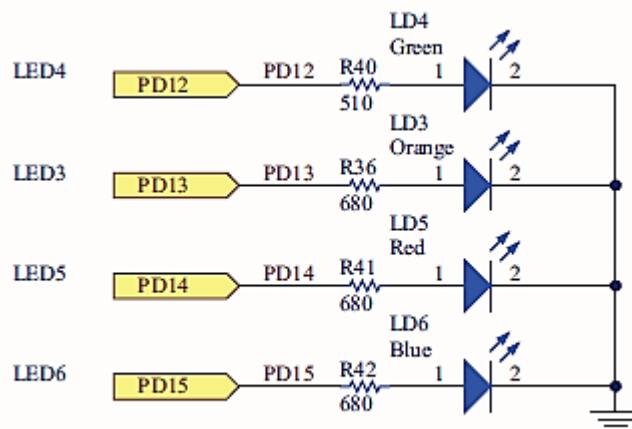
2.20.3 Digitális I/O kezelése, megszakítás és időzítő használata - Mintapélda

Az előző egyszerű mintapéldában láthattuk, hogy milyen lépéseket kell elvégeznünk ahhoz, hogy egy egyszerű, csak a mikrokontroller általános célú ki/bemeneteit használó alkalmazást elkészítsünk. Bővítsük ki a feladatunkat még néhány, minden mikrokontrollerben megtalálható periféria használatával.

Készítsünk egy olyan mikrokontrolleres alkalmazást, mely a fejlesztőkártyán körben elhelyezett 4 LED-en futófényt valósít meg, a futófény időzítését a mikrokontroller egy időzítő perifériájával valósítsuk meg, valamint készítsük fel az alkalmazást arra, hogy a futófény léptetése a fejlesztőkártyán található nyomógomb megnyomásával leállítható, ismételt megnyomásával pedig elindítható legyen.

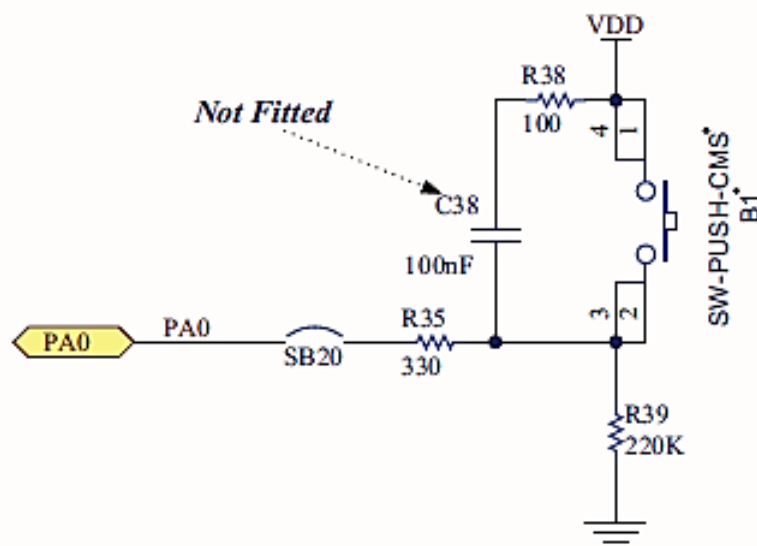
Csakúgy, mint az előző mintapélda esetében, most is a fejlesztőkártya kapcsolási rajzának vizsgálatával kezdjük a fejlesztést.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 24. oldal
--	--	---



9. ábra Az STM32F4 Discovery kapcsolási rajzának LED illesztést leíró része

A kapcsolási rajz LED illesztést leíró részét vesszük elő ismét, megnézzük rajta, hogy a 4 LED a mikrokontroller PD12-PD15 lábához csatlakozik, és mindegyik egyenes (ponált) logika szerint működik.



10. ábra Az STM32F4 Discovery kapcsolási rajzának nyomógomb illesztést leíró része

A kártya kapcsolási rajzában a következő – számunkra ebben a feladatban lényeges – része a nyomógomb illesztését leíró rész. Láthatjuk, hogy a nyomógomb a mikrokontroller PA0 lábához kapcsolódik (A port, 0-s láb), és ha megnyomjuk a gombot, akkor a mikrokontroller lábat felhúzza tápfeszültségre – azaz logikai 1 szintre.

Tehát a perifériáink kezeléséhez a mikrokontroller két portján – az A és D porton – lévő összesen 5 lábat kell vezérelnünk.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 25. oldal
--	--	---

Gondoljuk végig, hogy hogyan fogjuk felépíteni az alkalmazásunkat: első lépésben inicializálnunk kell a ki/bemeneti portokat – az előző feladatban megszerzett ismeretek alapján ez már nem okozhat problémát. Második lépésben az időzítő egységet kell felprogramoznunk úgy, hogy megfelelő időközönként megszakítást generáljon, amit majd lekezelve el tudjuk végezni a futófény léptetését. Harmadik lépésben a külső lábról érkező megszakítást kell engedélyeznünk, hogy a futófényünk gombnyomásra leállítható és elindítható legyen. Negyedik lépésben be kell állítanunk a megszakítás vezérlő alrendszerét, hogy a perifériáink által generált megszakításkérések érvényre juthassanak. Végezetül pedig el kell készítenünk a két megszakítás kezelő függvényünket, melyekben a futófény léptetését, valamint a megállítást és elindítást feladatát kell megvalósítanunk.

Az előző feladatban láttunk egy példát arra, hogy hogyan kell az általános célú ki/bemeneti portokat beállítani, ezek alapján készítsük el a bővített feladatunkhoz a portok beállítását végző kódrészletet.

```
GPIO_InitTypeDef GPIO_InitStructure;

/* GPIOD periféria órajel engedélyezés */
__GPIOD_CLK_ENABLE();

/* GPIOA periféria órajel engedélyezés */
__GPIOA_CLK_ENABLE();

/* GPIO lábak konfigurálása a LED-ekhez*/
GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP; /* kimenet, push-pull */
GPIO_InitStructure.Pull = GPIO_NOPULL; /* nincs felhúzó */
GPIO_InitStructure.Speed = GPIO_SPEED_LOW; /* kis sebesség */
GPIO_InitStructure.Pin = GPIO_PIN_12 | GPIO_PIN_13 |
                        GPIO_PIN_14 | GPIO_PIN_15; /* 12-15 -ös láb */
HAL_GPIO_Init(GPIOD, &GPIO_InitStructure);

GPIO_InitStructure.Mode = GPIO_MODE_IT_FALLING; /* Bemenet IT-vel */
GPIO_InitStructure.Pin = GPIO_PIN_0; /* 0. láb */
HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);

/* Egy LED bekapcsolása */
HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12, GPIO_PIN_SET);
```

Láthatjuk, hogy első lépésben mindkét port órajelét engedélyezzük – erre mindenképp szükségünk van, mert órajel hiányában a perifériák nem működnek. Második lépésben inicializáljuk a LED vezérlő lábakat, az egyetlen különbség az előző feladathoz képest annyi, hogy amikor a struktúrában a Pin tagot töltjük ki, akkor nem csak egyetlen lábat adunk meg, hanem négyet. A négy lábat azonosító konstanst vagy kapcsolattal szerepeltetjük, mivel a konstansok egy regiszter egyetlen bitjét reprezentálják.

A LED vezérlő lábak beállítása után áttérünk a nyomógomb által vezérelt láb konfigurálásához. Itt a lábat bemenetként kell használnunk – hiszen a nyomógomb egy beviteli periféria. Azt szeretnénk, hogy a nyomógomb megnyomásakor – lefutó élre – megszakítás generálódjon, és ezt a megszakítást lekezelve tudjuk a futófényünket

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 26. oldal
--	--	---

megállítani illetve elindítani. A GPIO_MODE_IT_FALLING beállítással ezt a működési módot tudjuk elérni.

Ezzel gyakorlatilag a ki/bemeneti lábak működési módját beállítottuk, áttérhetünk az alkalmazott időzítő beállítására. Mit csinál az időzítő, vagy az angol terminológia szerint Timer periféria? A mikrokontrollerek integrált perifériái fejezetben részletesen bemutatásra kerül a mikrokontrollerekben használt időzítő egységek belső felépítése, és működési módjaik, itt nekünk most csak a legegyszerűbb működési módra lesz szükségünk.

Az időzítő modulok legegyszerűbb működési módjukban egyszerű bináris számlálók, melyek a mikrokontroller órajelével arányosan számolnak, és amikor elérnek egy előre megadott értéket, akkor bebillentenek egy jelző flag-et és nullázzák magukat.

A mintapéldában mi egy 16 bites időzítőt fogunk használni, még hozzá felfelé számláló üzemmódban. Úgy állítjuk majd be az időzítőt, hogy 500msec gyakorisággal állítsa be a jelző flag-et.

A timer beállítását végző kódrészlet a következők szerint alakul:

```
/* Timer (TIM4) órajel engedélyezés */
__TIM4_CLK_ENABLE();

/* Timer konfigurálása */
Timer_Handle.Instance = TIM4;
Timer_Handle.Init.CounterMode = TIM_COUNTERMODE_UP;
Timer_Handle.Init.Period = 41999;
Timer_Handle.Init.Prescaler = 999;
HAL_TIM_Base_Init(&Timer_Handle);
```

Első lépésben, mint azt már a ki/bemeneteknél megszoktuk, engedélyeznünk kell az időzítő modul órajelét.

Következő lépésben – mint a ki/bemeneteknél – egy az időzítő beállításait leíró struktúrát töltünk ki. A struktúrának négy – számunkra ebben a feladatban fontos – tagja van. Az Instance, a CounterMode, a Period és a Prescaler. Ezekkel állítjuk be, hogy hogyan működjön a kiválasztott időzítőnk. Láthatjuk, hogy felfelé számláló módot állítunk be, Előosztónak 999-et, Periódusnak pedig 41999-et állítunk be. A struktúra kitöltése után meghívjuk az időzítő inicializáló függvényét – HAL_TIM_Base_Init – amely a megadott beállításokkal ebben az egyszerű üzemmódban inicializálja az időzítőt.

Hogyan számítható ki a Prescaler és a Period tagok értéke, mit határoznak ezek meg? A prescaler – azaz az előosztó azt szabja meg, hogy az időzítő a buszfrekvenciához képest milyen frekvenciával fog számolni, a periódus pedig a számláló végértékét adja meg, értékük a következő összefüggésekből számolható:

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 27. oldal
--	--	---

$$PRESALER = \frac{APBx_clk}{TIM_Counter_clk} - 1$$

$$PERIOD = \frac{TIM_Counter_clk}{f} - 1$$

Nézzük, esetünkben hogyan alakulnak ki ezek az értékek. A mikrokontroller dokumentációjából tudjuk, hogy az időzítőnk az APB1 perifériabuszra csatlakozik, ezen a buszon az időzítőnk alap órajele 84MHz, azt is tudjuk, hogy 2Hz-es gyakorisággal – 500 msec periódussal – szeretnénk, ha az időzítőnk jelezne. Láthatjuk, hogy szükségünk van még egy paraméterre ahhoz, hogy az előosztó és a periódus értékét meghatározhassuk (3 ismeretlen, és két egyenlet). A harmadik ismeretlen paramétert – az időzítőben lévő számláló órajelét szabadon meghatározhatjuk, azonban erre is léteznek szokások. Általában az időzítő órajelét úgy szoktuk beállítani, hogy kerek frekvenciával számoljon (pl. 1kHz). Nézzük meg, hogy mit kellene az előosztóba beállítanunk ahhoz, hogy az időzítőnk 1kHz-el számoljon:

$$PRESALER = \frac{APBx_clk}{TIM_Counter_clk} - 1 = \frac{84000000}{1000} - 1 = 83999$$

Az összefüggés alapján 83999-et kellene az előosztóba írunk ahhoz, hogy az időzítőnk 1kHz-es frekvenciával számoljon. Mi mégsem ezt az értéket használtuk az időzítő beállításánál, mi lehet ennek az oka? Az időzítőnk, mint azt már korábban említettük 16 bites, ez azt is jelenti, hogy az előosztónk regisztere is 16 bites, 16 biten pedig a legnagyobb ábrázolható szám a 65535. Tehát a kiszámított előosztó érték egyszerűen nem fér bele az előosztó regiszterébe, másik előosztót kell választanunk! Ha az előosztót nem tudjuk beállítani úgy, hogy az időzítőnk kerek frekvenciával számoljon, akkor az előosztó értékét úgy szoktuk megválasztani, hogy az időzítő frekvenciája tized- század-ezred része legyen a buszfrequenciának. Mi most az ezres leosztást választjuk, ehhez 999-et kell az előosztóba írunk.

$$PRESALER = \frac{APBx_clk}{TIM_Counter_clk} - 1 = \frac{84000000}{84000} - 1 = 999$$

A számláló órajel frekvenciájának ismeretében már a periódusidő is számítható.

$$PERIOD = \frac{TIM_Counter_clk}{f} - 1 = \frac{84000}{2} - 1 = 41999$$

Láthattuk tehát, hogy hogyan kell beállítanunk az időzítő paramétereit. Következő lépésben a külső vonalról érkező megszakításkérésrel kell foglalkoznunk. Azt szeretnénk, ha a nyomógombunk lenyomásakor egy megszakítás generálna, amelyre reagálva le tudjuk állítani, illetve el tudjuk indítani a futófényünket.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 28. oldal
--	--	---

Ezek után nézzük a megszakítás vezérlő beállítására szolgáló kódrészletet.

```
/* NVIC Prioritás modell beállítása */
HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_1);

/* Timer interrupt prioritásának beállítása */
HAL_NVIC_SetPriority(TIM4_IRQn, 0, 0);

/* Timer interrupt engedélyezése */
HAL_NVIC_EnableIRQ(TIM4_IRQn);

/* Timer elindítása */
HAL_TIM_Base_Start_IT(&Timer_Handle);
```

Első lépésben kiválasztjuk a prioritásmodellt, tehát megmondjuk, hogy hány prioritás csoportunk legyen – mi olyan beállítást választunk, hogy két csoportunk legyen. Ezek után a HAL_NVIC_SetPriority függvény meghívásával beállítjuk a TIM4-es időzítő megszakítás prioritását, majd a HAL_NVIC_EnableIRQ függvénnyel engedélyezzük a megszakítás érvényre jutását. Láthatjuk, hogy a lehető legmagasabb prioritást rendeltük hozzá az időzítő megszakításához. Ezek után elindítjuk az időzítőt a HAL_Base_Start_IT függvénnyel. Az elindításhoz használt függvény határozza meg, hogy az időzítőnk próbál-e egyáltalán megszakítást kérni. Amennyiben az időzítőnk a HAL_TIM_Base_Start függvénnyel indítanánk el, akkor az időzítő csak a túlsordulást jelző flag-et billentené be, megszakítást nem kezdeményezne.

A külső megszakításunk engedélyezése is hasonlóképpen zajlik, hozzárendelünk egy prioritást a megszakításunkhoz, illetve engedélyezzük azt az előzőekben is használt függvényekkel.

```
/* Gomb interrupt prioritásának beállítása */
HAL_NVIC_SetPriority(EXTI0_IRQn, 1, 0);

/* Gomb interrupt engedélyezése */
HAL_NVIC_EnableIRQ(EXTI0_IRQn);
```

Az alkalmazás inicializációs részével el is készültünk, már „csak” a megszakításokat kell lekezelnünk – ha jobban belegondolunk ez lesz az alkalmazásunk tényleges funkciót megvalósító része.

Elsőként az időzítő megszakítását kezeljük.

```
void TIM4_IRQHandler(void)
{
    HAL_TIM_IRQHandler(&Timer_Handle);
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    uint16_t LedVal;
    uint16_t PortVal;
```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 29. oldal
--	--	---

```

if ( htim->Instance == TIM4 )
{
    PortVal = LedVal = GPIOD->ODR;
    PortVal &= 0x0fff;
    LedVal &= 0xf000;
    LedVal <<= 1;
    if (LedVal == 0)
        LedVal = 0x1000;
    PortVal |= LedVal;
    if (LedEnable)
        GPIOD->ODR = PortVal;
}
}

```

A natív – használt periféria könyvtártól független – megszakítás kezelő függvényben meghívjuk a függvénykönyvtárunk beépített megszakítás kezelő függvényét. Ez azzal az előnnyel jár számunkra, hogy a megszakítás jelző flag állapotának vizsgálatát és a flag törlését a könyvtári függvény elvégzi helyettünk, majd pedig meghívja az adott megszakítás típushoz tartozó callback függvényt. A callback függvény a legtöbb függvénykönyvtárban olyan függvényt jelent, melyet a fejlesztőnek kell megírnia, és a függvénykönyvtár megfelelő függvényei ezt a megírt callback függvényt hívják meg.

A callback függvényben létrehozunk két lokális 16 bites változót, az egyiket a port értékének tárolására, a másikat pedig a LED-ek léptetésére fogjuk használni. A callback függvényben ezek után megvizsgáljuk, hogy melyik időzítő megszakításának hatására hívódott meg a függvényünk. Esetünkben csak a TIM4 időzítő okozhatta, de bonyolultabb alkalmazásoknál a mikrokontroller több időzítőjét is használhatjuk, melyek megszakítása esetén szintén ugyanez a callback függvény kerülne meghívásra. Amennyiben tényleg TIM4 időzítő újratöltése okozta a megszakítást, akkor be kell olvasnunk a LED-eket vezérlő port értékét, mégpedig a PortVal és a LedVal változókba. A port aktuális állapotát a kimeneti adatregiszter (ODR) értékének kiolvasásával tudhatjuk meg – ehhez nincs a függvénykönyvtárban külön függvény. A portVal változó értékét alapul véve maszkolással előállítjuk a LED-jeink értékét a LedVal változó felső négy bitjében, illetve a PortVal változónak a felső 4 bitjét kitöröljük – mert ide fogjuk beírni a LED-ek következő állapotát. Következő lépésben balra léptetjük a LedVal változónkat eggyel, és megvizsgáljuk, hogy maradt-e még egyes benne, ha nem, akkor visszatöltjük a 4 bit közül a legalsóba az egyest. Következő lépésben pedig beírjuk a módosított LED értékeket a PortVal változóba. Utolsó lépésként amennyiben a globális engedélyező változónk az engedélyezés állapotban van, frissítjük a port kimeneti adatregiszterét – azaz átállítjuk a LED-ek állapotát.

Megfigyelhetjük, hogy a porton csak azoknak a lábakkal az értékét változtattuk meg, amelyekre a LED-ek csatlakoztatva vannak, a többi lábat változatlanul hagytuk – a gyakorlatban ezt mindig így kell csinálni! Amennyiben nem így teszünk – tehát több láb állapotát változtatjuk a ténylegesen szükségesnél – egy későbbi továbbfejlesztés során könnyen olyan rejtélyes hibákba ütközhetünk, melyeket több óra, vagy akár több napos hibakeresés árán tudunk csak esetleg megjavítani.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 30. oldal
--	--	---

Az alkalmazásunkból már csak egyetlen megszakítás kezelő rutin megírása maradt, mégpedig a nyomógomb külső megszakításának kezelése.

```
FunctionalState LedEnable = ENABLE;
void EXTI0_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
}

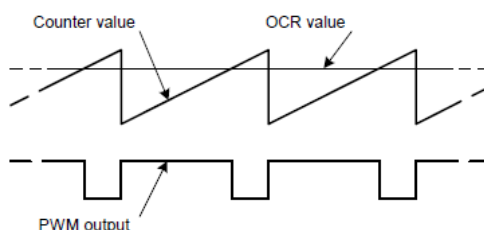
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if ( GPIO_Pin & GPIO_PIN_0 )
    {
        LedEnable = !LedEnable;
    }
}
```

A megszakítás kezelő rutinunk és callback függvényünk felépítése nagyon hasonlít az előzőhöz. A callback függvényben megvizsgáljuk, hogy tényleg az EXTI0 vonalon bekövetkezett felfutó él okozta-e a megszakítást. Amennyiben igen, akkor az LedEnable globális változó értékétől függően letiltjuk, vagy engedélyezzük az a LED-ek állapotának módosítását.

2.20.4 Digitális I/O kezelése, időzítő használata PWM módban - Mintapélda

Az eddig mintapéldáinkban a fejlesztőkártyán található LED-eket viszonylag ritkán kapcsoltuk ki ill. be. Bekapcsolt állapotukban teljes fényerővel világítottak. Következő példánkban szeretnénk megvalósítani, hogy a fejlesztőkártyán található LED-ek különböző fényerővel világítsanak. Ha a LED-eket különböző kitöltési tényezőjű négyyszögjellel hajtjuk meg, akkor a szemünk tehetetlensége miatt azt fogjuk észlelni, hogy a LED-ek fényereje a négyyszögjel kitöltési tényezőjével arányosan változni fog.

A mikrokontrollerek integrált perifériái részben részletesen tanultunk az integrált időzítő perifériákról és működési módjaikról. Az időzítő perifériánkat PWM üzemmódban használva a LED-ek vezérléséhez szükséges négyyszögjel előállítható.



11. ábra PWM jel

Az STM32F407 mikrokontrollerbe integrált TIM4 időzítő 4 kimeneti csatornával rendelkezik, melyek a mikrokontroller PD12-15 lábaira vezethető ki. Szerencsés helyzetben vagyunk, hiszen a PD12-15 lábakra vannak a LED-jeink is csatlakoztatva, így a generált PWM jellel meg tudjuk majd hajtani őket.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 31. oldal
--	--	---

Ahhoz, hogy az időzítő periféria által generált jelek kerüljenek ki a PD12-15-ös lábakra, a GPIO lábakra alternatív funkciót kell konfigurálnunk. Ez azt jelenti, hogy ezek a lábak nem GPIO módban fognak működni, hanem valamely integrált periféria jelei fognak megjelenni rajtuk.

```
GPIO_InitTypeDef GPIO_InitStruct;

/* GPIOD periféria órajel engedélyezés */
__GPIO_CLK_ENABLE();
/* kimenet, push-pull, alternate */
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_LOW;
GPIO_InitStruct.Pin =
    GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 | GPIO_PIN_15;

/* Alternatív funkció kiválasztása */
GPIO_InitStruct.Alternate = GPIO_AF2_TIM4;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
```

Az alternatív funkció kiválasztásához egyrészt az inicializációs struktúra mód tagját kell beállítanunk a GPIO_MODE_AF_PP konstans kiválasztásával – ezzel azt érjük el, hogy nem GPIO módban fognak működni ezek a lábak. Az inicializáló struktúra Alternate tagjának a GPIO_AF2_TIM4 értéket beállítva pedig azt állítjuk be, hogy melyik periféria jelei jelenjenek meg a GPIO funkció helyett.

A PWM jelnek két fontos paramétere van, az egyik a jel frekvenciája, a másik pedig a kitöltési tényező. A generált jel frekvenciájának beállítása az előző mintapéldával megegyező módon a timer leíró struktúra Init tagjában történik.

```
TIM_HandleTypeDef Timer_Handle;

/* Timer (TIM4) órajel engedélyezés */
__TIM4_CLK_ENABLE();

/* Timer konfigurálása */
Timer_Handle.Instance = TIM4;
Timer_Handle.Init.CounterMode = TIM_COUNTERMODE_UP;
Timer_Handle.Init.Period = 8399;
Timer_Handle.Init.Prescaler = 0;
Timer_Handle.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;

HAL_TIM_PWM_Init(&Timer_Handle);
```

Mivel az előosztót 0-ra állítottuk, a periódus értékét pedig 8399-re, és mivel az időzítőnk órajel frekvenciája 84 MHz, így a PWM jelünk frekvenciája 10 kHz lesz.

A struktúra kitöltése után a HAL_TIM_PWM_Init függvényhívással PWM módban inicializáljuk az időzítő perifériánkat.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 32. oldal
--	--	---

Következő lépésben a kitöltési tényezőt kell beállítanunk, illetve el kell indítanunk a PWM jelek generálását csatornánként.

```
TIM_OC_InitTypeDef PWM_InitStruct;

PWM_InitStruct.OCMode = TIM_OCMode_PWM1;
PWM_InitStruct.OCpolarity = TIM_OCPolarity_HIGH;
PWM_InitStruct.OCFastMode = TIM_OCFAST_DISABLE;
/* Duty cycle = 100 % */
PWM_InitStruct.Pulse = 8399;
HAL_TIM_PWM_ConfigChannel(&Timer_Handle, &PWM_InitStruct,
TIM_CHANNEL_1);
/* Duty cycle = 50 % */
PWM_InitStruct.Pulse = 4199;
HAL_TIM_PWM_ConfigChannel(&Timer_Handle, &PWM_InitStruct,
TIM_CHANNEL_2);
/* Duty cycle = 25 % */
PWM_InitStruct.Pulse = 2099;
HAL_TIM_PWM_ConfigChannel(&Timer_Handle, &PWM_InitStruct,
TIM_CHANNEL_3);
/* Duty cycle = 12,5 % */
PWM_InitStruct.Pulse = 1049;
HAL_TIM_PWM_ConfigChannel(&Timer_Handle, &PWM_InitStruct,
TIM_CHANNEL_4);
/* Timer elindítása */
HAL_TIM_PWM_Start(&Timer_Handle, TIM_CHANNEL_ALL );
```

A kitöltési tényezőt a PWM_InitStruct Pulse tagja fogja meghatározni, mégpedig úgy, hogy a kitöltési tényező a Pulse és az időzítő beállításánál megadott Period Hányadosa lesz.

2.20.5 Az STM32 mikrokontroller inicializálása

Az eddigi mintapéldáink azt feltételezték, hogy a mikrokontrollerünk egy rendesen inicializált állapotban jut el a main függvény végrehajtásáig. Ez teljesen jogos feltételezés egy komoly fejlesztőkörnyezettel generált alkalmazásnál, viszont az inicializálás nem önmagától történik, hanem azt valakinek – esetünkben a fejlesztőeszköz gyártójának, vagy nekünk – el kell azt készítenie.

Mit értünk inicializált állapot alatt?

- A RAM tartalma inicializálva van
 - globális változók kezdőértéke be van töltve
 - inicializálatlan globális változóink 0 értékre vannak állítva
 - a heap és a stack inicializálva van
- A megszakítási vektortáblánk be van állítva
- A mikrokontroller órajele a kívánt értékre van beállítva
- A mikrokontroller konfigurációs regiszterei megfelelően be vannak állítva
- Az alkalmazáshoz használt függvénykönyvtár inicializálva van

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 33. oldal
--	--	---

Mint láthatjuk az inicializálás során elég sok feladatot el kell végezni, és az sem mindegy hogy mennyire hatékonyan valósítjuk ezt meg.

A mikrokontroller inicializálása a startup kódban történik meg. Az STM32CubeIDE fejlesztőrendszerben generált projektben két fájlban (system_stm32f4xx.c és startup_stm32f407xx.s) találhatóak azok a függvények és assembly szubrutinok, melyek az inicializálást végzik. A startup rutinok működése feltételezi, hogy az alkalmazás fordítása során használt linker konfigurációs fájl (linker script) létrehoz bizonyos szimbólumokat, illetve specifikálja a mikrokontroller különböző memóriatartományainak kezdőcímét, méretét és felépítését.

Az alábbi script részlet az STM32F407 kontrollerhez készült linker scriptből származik.

```

/* Entry Point */
ENTRY(Reset_Handler)
/* Highest address of the user mode stack */
_estack = 0x20020000; /* end of 128K RAM on AHB bus*/
/* Generate a link error if heap and stack don't fit into RAM */
_Min_Heap_Size = 0; /* required amount of heap */
_Min_Stack_Size = 0x400; /* required amount of stack */
/* Specify the memory areas */
MEMORY
{
    FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 1024K
    RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 192K
    MEMORY_B1 (rx)  : ORIGIN = 0x60000000, LENGTH = 0K
}

```

A fenti script részlet első sora megadja, hogy melyik szubrutin lesz az alkalmazás tényleges belépési pontja – azaz mi lesz a startup kódunkat megvalósító szubrutin neve (Reset_Handler).

Ezt követően a stack és a heap minimális méretét és helyét adja meg a script. Ha a fordítás során kiderül, hogy a lefoglalt stack terület nem elegendő – például a függvényhívási láncolatokhoz, illetve lokális változókhoz – akkor a fordító hibát tud jelezni.

Végezetül pedig a mikrokontroller különböző memóriatartományainak tulajdonságait: kezdőcímét, méretét, hozzáférési módját specifikáljuk. A linker hibát tud jelezni abban az esetben is, ha például olyan nagy alkalmazást próbálunk lefordítani ami nem fér bele a mikrokontroller flash memóriájába, illetve ha az adataink nem férnek el a RAM-ban.

A linker script az alábbi ábrán látható struktúrával foglalja le a RAM területét. A RAM elején az inicializált globális változóink kerülnek elhelyezésre, utána következnek az inicializálatlan változók a BSS szegmensben, majd a heap, legvégül a stack. A stack a RAM végén kezdődik, újabb elemek stack-re helyezésével a csökkenő címek felé tölti a RAM-ot.

Address	Segment type
0x20000000	Data
	BSS
↓	Heap
↑	Stack

12. ábra STM32 alkalmazás RAM felépítése

Ahogy a linker script-ben is láthattuk az alkalmazásunk tényleges belépési pontja a Reset_Handler szubrutin. Az alábbiakban láthatjuk ennek megvalósítását.

```
.weak Reset_Handler
.type Reset_Handler, %function
Reset_Handler:
    ldr    sp, _estack    /* set stack pointer */

/* Copy the data segment initializers from flash to SRAM */
    movs  r1, #0
    b     LoopCopyDataInit
CopyDataInit:
    ldr  r3, _sidata
    ldr  r3, [r3, r1]
    str  r3, [r0, r1]
    adds r1, r1, #4

LoopCopyDataInit:
    ldr  r0, _sdata
    ldr  r3, _edata
    adds r2, r0, r1
    cmp  r2, r3
    bcc  CopyDataInit
    ldr  r2, _sbss
    b     LoopFillZerobss
/* Zero fill the bss segment. */
FillZerobss:
    movs  r3, #0
    str  r3, [r2], #4

LoopFillZerobss:
    ldr  r3, _ebss
    cmp  r2, r3
    bcc  FillZerobss

/* Call the clock system initialization function.*/
    bl   SystemInit
/* Call static constructors */
    bl   __libc_init_array
/* Call the application's entry point.*/
    bl   main
    bx   lr
.size Reset_Handler, .-Reset_Handler
```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 35. oldal
--	--	---

A gcc egy speciális szolgáltatása, hogy ún. weak függvényeket tudunk készíteni mind assemblyben, mind pedig C-ben. A weak direktívával deklarált függvény felüldefiniálható. Ez azt jelenti, hogy mindaddig a weak direktívával deklarált függvényt fogja a fordító az alkalmazásunkba fordítani, amíg nem készítünk egy ugyanilyen nevű viszont weak nélkül definiált függvényt. Ezzel a weak módosítóval került deklarálásra a Reset_Handler is – tehát nyugodtan írhatunk sajátot, ha szeretnénk és nem kell módosítanunk az eredeti implementációt. Ugyanezzel a weak módosítóval készít default implementációt a callback függvényekhez a HAL library is.

A Reset_Handler – ahogy azt a fenti kódrészletből is láthatjuk – először a stack pointert inicializálja: beállítja a RAM utolsó címére. Ezek után a Data szegmens inicializálása következik – átmásolja az inicializált globális változóink értékét a flashból a RAM-ba. A Data szegmens inicializálását a BSS szegmens inicializálása követi – nullákkal tölti fel a BSS szegmens minden bájtyát.

Ezek után a SystemInit függvény meghívása következik – ez már egy C nyelven elkészített függvény, mely a system_stm32f4xx.c állományban található. A SystemInit függvény a mikrokontroller alapvető működéséhez szükséges regisztereket állítja alap állapotba, és beállítja a mikrokontroller órajelét. Az órajel beállítását végző kód olyan módon állítja be az órajelrendszert, hogy belső oszcillátorról működjön 16MHz-en (a mikrokontroller maximális órajelfrekvenciája 168 MHz lehet, tehát az alap órajel beállító rutin nem hozza ki a maximumot a mikrokontrollerből).

Mivel a gcc nem csak C nyelvű alkalmazásokat képes fordítani, hanem C++ nyelven elkészítetteket is, így gondoskodnia kell a startup kódnak arról is, hogy a globális C++ objektumaink statikus konstruktorai meghívódjanak. A statikus konstruktorok meghívása után jutunk el az inicializálás folyamatában oda, hogy meghívhatjuk a main függvényt.

A startup_stm32f407xx.s fájl a Reset_Handler-en kívül tartalmazza még a megszakítási vektortáblát is. Az alábbiakban egy részletet láthatunk ennek a definíciójából.

```
.section .isr_vector,"a",%progbits
.type g_pfnVectors, %object
.size g_pfnVectors, .-g_pfnVectors

g_pfnVectors:
.word _estack
.word Reset_Handler
.word NMI_Handler
.word HardFault_Handler
.word MemManage_Handler
/* ... */
/* External Interrupts */
.word WWDG_IRQHandler /* Window WatchDog
*/
.word PVD_IRQHandler /* PVD through EXTI Line
detection */
.word TAMP_STAMP_IRQHandler /* Tamper and TimeStamps through
/* ... */
```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 36. oldal
--	--	---

A megszakítási vektortábla minden eleméhez hozzárendelésre kerül egy kezelőfüggvény is, mégpedig weak direktívával – hogy a tényleges megszakításkezelőink elkészíthetők legyenek. Alapértelmezett megszakításkezelőként a Default_Handler kerül hozzárendelésre a vektortábla minden eleméhez. A Default_Handler egy egyszerű végtelen ciklust futtat, így ha olyan megszakítás következik be, amihez nem írtuk meg a tényleges megszakításkezelőt, beugrik ebbe a végtelen ciklusba, és ott is marad – ezt az állapotot debuggerben könnyedén el lehet kapni.

```
.weak      NMI_Handler
.thumb_set NMI_Handler,Default_Handler

.weak      HardFault_Handler
.thumb_set HardFault_Handler,Default_Handler

/* ... */
.weak      WWDG_IRQHandler
.thumb_set WWDG_IRQHandler,Default_Handler

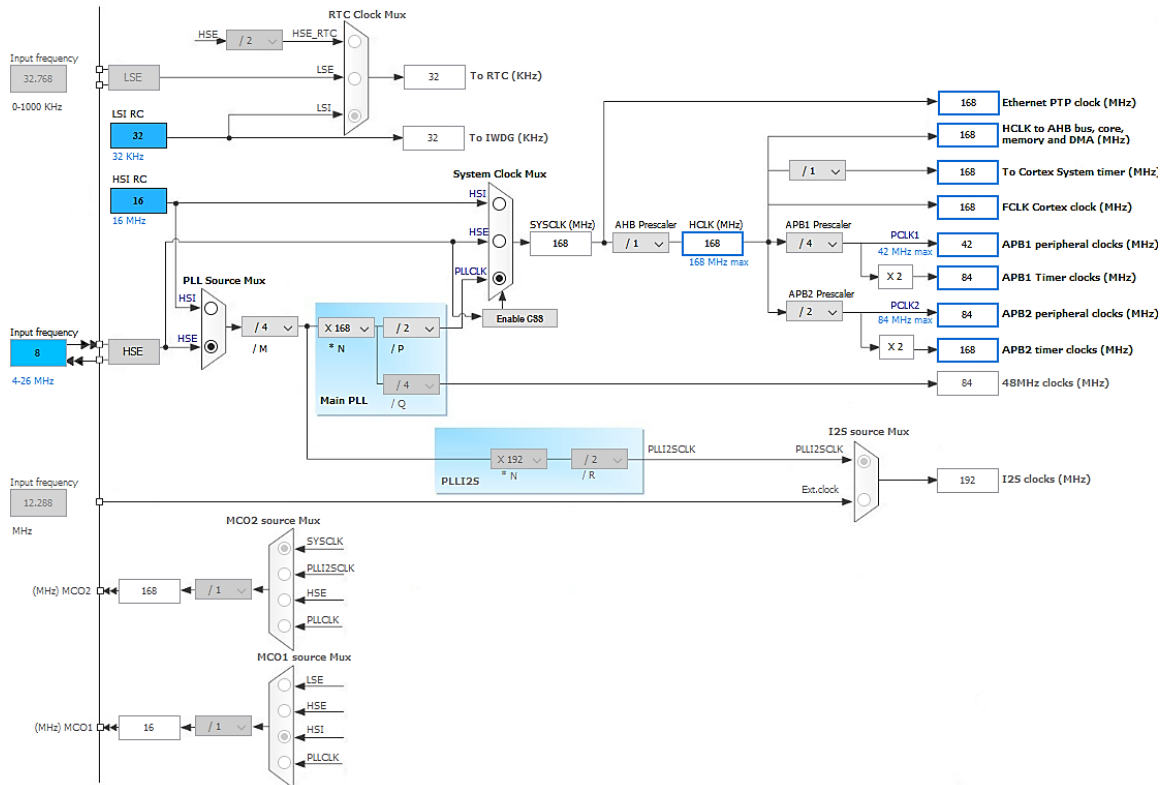
.weak      PVD_IRQHandler
.thumb_set PVD_IRQHandler,Default_Handler

/* ... */
.section .text.Default_Handler,"ax",%progbits
Default_Handler:
Infinite_Loop:
    b Infinite_Loop
.size Default_Handler,.-Default_Handler
```

Mint az a korábbiakban említettem az alap inicializáló függvények alacsony – 16 MHz-es – órajelet állítanak csak be. Ha maximálisan ki szeretnénk használni a mikrokontrollerünk képességeit, akkor ezt mindenképp módosítanunk szükséges.

A mikrokontroller dokumentációja részletesen leírja a belső órajel struktúra felépítését, és konfigurálását. Mivel az órajelrendszer önmagában is elég bonyolult, és számos beállítási lehetőséggel rendelkezik, így legegyszerűbben úgy tudjuk az órajelet átállítani, ha segédprogramot használunk hozzá. Az ST Microelectronics készített egy konfigurációs segédprogramot – STM32 CubeMX – amivel többek között az órajel beállításait tudjuk egy grafikus felületen keresztül elvégezni. A szoftver a beállításoknak megfelelően képes generálni számunkra az órajel beállító függvényt – melyet a projektünkbe bemásolhatunk és ezt a függvényt meghívva a beállított órajelfrekvenciával fog működni a mikrokontrollerünk.

Az alábbi ábrán láthatjuk az STM32 CubeMX szoftver órajel konfigurációhoz használható felületét.



13. ábra STM32 CubeMX órajel beállítás

2.20.6 SPI kommunikáció – Mintapélda

A következő mintapéldák az ST Microelectronics saját – a fejlesztőkártyához kiadott mintapéldái, melyeken keresztül betekintést nyerhetünk az SPI kommunikáció különböző lehetséges megvalósítási lehetőségeibe. A mintapélda azt feltételezi, hogy két teljesen egyforma STM32F4 Discovery fejlesztőkártyát az SPI2 kommunikációs periféria PB13-15 vonalait összekötjük. A két kártya közül az egyik lesz az SPI Master, a másik pedig a slave. A kommunikáció a master kártyán történő gombnyomásra indul el, és mindkét egység összehasonlítja a kapott adatokat a saját adataival, ha ezek nem egyeznek, akkor azt a LED-eken jelzik.

A mintapéldában készen kapunk egy BSP (Board Support Package) szoftvercsomagot, melyben a fejlesztőkártyán megtalálható összes periféria alacsony szintű kezelése meg van valósítva. Erre a BSP rétegre építkezve az alkalmazáslogika jól hordozható, újrafelhasználható tud lenni.

Első lépésben megvizsgáljuk ennek a BSP rétegnek a felépítését, működését.

```
typedef enum
{
    LED4 = 0,
    LED3 = 1,
    LED5 = 2,
```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 38. oldal
--	--	---

```

    LED6 = 3
} Led_TypeDef;

typedef enum
{
    BUTTON_KEY = 0,
} Button_TypeDef;

typedef enum
{
    BUTTON_MODE_GPIO = 0,
    BUTTON_MODE_EXTI = 1
} ButtonMode_TypeDef;

#define LEDn                                4
#define LED4_PIN                            GPIO_PIN_12
#define LED4_GPIO_PORT                    GPIOD
#define LED4_GPIO_CLK_ENABLE()             __GPIOD_CLK_ENABLE()
#define LED4_GPIO_CLK_DISABLE()           __GPIOD_CLK_DISABLE()
#define LED3_PIN                            GPIO_PIN_13
#define LED3_GPIO_PORT                    GPIOD
#define LED3_GPIO_CLK_ENABLE()             __GPIOD_CLK_ENABLE()
#define LED3_GPIO_CLK_DISABLE()           __GPIOD_CLK_DISABLE()

#define LED5_PIN                            GPIO_PIN_14
#define LED5_GPIO_PORT                    GPIOD
#define LED5_GPIO_CLK_ENABLE()             __GPIOD_CLK_ENABLE()
#define LED5_GPIO_CLK_DISABLE()           __GPIOD_CLK_DISABLE()
#define LED6_PIN                            GPIO_PIN_15
#define LED6_GPIO_PORT                    GPIOD
#define LED6_GPIO_CLK_ENABLE()             __GPIOD_CLK_ENABLE()
#define LED6_GPIO_CLK_DISABLE()           __GPIOD_CLK_DISABLE()

GPIO_TypeDef* GPIO_PORT[LEDn] = {LED4_GPIO_PORT,
                                  LED3_GPIO_PORT,
                                  LED5_GPIO_PORT,
                                  LED6_GPIO_PORT};

const uint16_t GPIO_PIN[LEDn] = {LED4_PIN,
                                  LED3_PIN,
                                  LED5_PIN,
                                  LED6_PIN};

#define LEDx_GPIO_CLK_ENABLE( __INDEX__ )
do{if((__INDEX__) == 0) LED4_GPIO_CLK_ENABLE(); else \
    if((__INDEX__) == 1) LED3_GPIO_CLK_ENABLE(); else \
    if((__INDEX__) == 2) LED5_GPIO_CLK_ENABLE(); else \
    if((__INDEX__) == 3) LED6_GPIO_CLK_ENABLE(); \
}while(0)

#define BUTTONn                            1
GPIO_TypeDef* BUTTON_PORT[BUTTONn] = {KEY_BUTTON_GPIO_PORT};
const uint16_t BUTTON_PIN[BUTTONn] = {KEY_BUTTON_PIN};
const uint8_t BUTTON_IRQn[BUTTONn] = {KEY_BUTTON_EXTI_IRQn};

```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 39. oldal
--	--	---

A BSP réteg definíciós fájljában számos előredefiniált típust és makrót találhatunk – melyet a C nyelv typedef és #define kulcsszavaival valósítanak meg. Láthatjuk, hogy egyrészt definiálásra kerülnek felsorolás típusok (Led_TypeDef, Button_TypeDef, ButtonMode_TypeDef), másrészt makrók (pl. LED4_PIN, LED6_GPIO_PORT, stb.). Ezekkel a definíciókkal azt lehet elérni, hogy az alkalmazáslogikába ne kelljen több helyre fixen beírni pl. portok neveit, láb sorszámozásokat, stb. Ez azért tud nagyon előnyös lenni, mivel ha egy hasonló, de más lábkiosztású eszközre szeretnénk majd a későbbiekben az alkalmazást átültetni, akkor csak a definíciós fájl tartalmát kell módosítanunk, minden egyéb része az alkalmazásunknak változatlan maradhat.

A LED-ek inicializálását és állapotmódosító függvényeinek megvalósítását láthatjuk az alábbi kódrészletben, láthatjuk, hogy pont ugyanazokban a lépésekben inicializálják és használják a GPIO portot, ahogyan azt mi is megtettük korábbi mintapéldáinkban.

```
void BSP_LED_Init(Led_TypeDef Led)
{
    GPIO_InitTypeDef GPIO_InitStructure;

    /* Enable the GPIO_LED Clock */
    LEDx_GPIO_CLK_ENABLE(Led);
    /* Configure the GPIO_LED pin */
    GPIO_InitStructure.Pin = GPIO_PIN[Led];
    GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStructure.Pull = GPIO_PULLUP;
    GPIO_InitStructure.Speed = GPIO_SPEED_FAST;

    HAL_GPIO_Init(GPIO_PORT[Led], &GPIO_InitStructure);

    HAL_GPIO_WritePin(GPIO_PORT[Led], GPIO_PIN[Led], GPIO_PIN_RESET);
}
void BSP_LED_On(Led_TypeDef Led)
{
    HAL_GPIO_WritePin(GPIO_PORT[Led], GPIO_PIN[Led], GPIO_PIN_SET);
}
```

A nyomógombot inicializáló függvények is nagyon hasonlóak ahhoz, amit a korábbi mintapéldákban mi is elkészítettünk. A különbség csupán annyi, hogy ez a függvény paraméterként várja a nyomógomb nevét a definiált felsorolás típus alapján, illetve a nyomógomb kívánt működési módját is.

```
void BSP_PB_Init(Button_TypeDef Button, ButtonMode_TypeDef Mode)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    /* Enable the BUTTON Clock */
    BUTTONx_GPIO_CLK_ENABLE(Button);
    if (Mode == BUTTON_MODE_GPIO)
    {
        /* Configure Button pin as input */
        GPIO_InitStructure.Pin = BUTTON_PIN[Button];
        GPIO_InitStructure.Mode = GPIO_MODE_INPUT;
        GPIO_InitStructure.Pull = GPIO_NOPULL;
        GPIO_InitStructure.Speed = GPIO_SPEED_FAST;
    }
}
```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 40. oldal
--	--	---

```

    HAL_GPIO_Init(BUTTON_PORT[Button], &GPIO_InitStruct);
}
if (Mode == BUTTON_MODE_EXTI)
{
    /* Configure Button pin as input with External interrupt */
    GPIO_InitStruct.Pin = BUTTON_PIN[Button];
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FAST;
    GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
    HAL_GPIO_Init(BUTTON_PORT[Button], &GPIO_InitStruct);
    /* Enable and set Button EXTI Interrupt to the lowest priority */
    HAL_NVIC_SetPriority((IRQn_Type)(BUTTON_IRQn[Button]), 0x0F, 0);
    HAL_NVIC_EnableIRQ((IRQn_Type)(BUTTON_IRQn[Button]));
}
}

```

A BSP réteg az SPI kommunikációs csatornára vonatkozóan is ugyanazt a koncepciót alkalmazza, mint a nyomógomboknál és a LED-eknél – azaz minden az SPI kommunikáció inicializálásához szükséges jelhez makrókat definiál.

```

#define SPIx                               SPI2
#define SPIx_CLK_ENABLE()                  __HAL_RCC_SPI2_CLK_ENABLE()
#define SPIx_SCK_GPIO_CLK_ENABLE()        __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MISO_GPIO_CLK_ENABLE()        __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MOSI_GPIO_CLK_ENABLE()        __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_FORCE_RESET()                 __HAL_RCC_SPI2_FORCE_RESET()
#define SPIx_RELEASE_RESET()               __HAL_RCC_SPI2_RELEASE_RESET()

#define SPIx_SCK_PIN                        GPIO_PIN_13
#define SPIx_SCK_GPIO_PORT                 GPIOB
#define SPIx_SCK_AF                         GPIO_AF5_SPI2
#define SPIx_MISO_PIN                      GPIO_PIN_14
#define SPIx_MISO_GPIO_PORT                 GPIOB
#define SPIx_MISO_AF                       GPIO_AF5_SPI2
#define SPIx_MOSI_PIN                      GPIO_PIN_15
#define SPIx_MOSI_GPIO_PORT                 GPIOB
#define SPIx_MOSI_AF                       GPIO_AF5_SPI2

```

Az SPI perifériához tartozó mikrokontroller lábak inicializálása nagyban hasonlít a PWM mintapéldában látottakhoz. Első lépésben engedélyezni kell a CLK, MISO, MOSI jelekhez tartozó portok órajelét, majd a lábak konfigurálásánál alternatív funkciót kell beállítani az egyes lábakhoz. Nem szabad megfeledkeznünk arról sem, hogy az SPI periféria órajelét engedélyezzük.

```

void HAL_SPI_MspInit(SPI_HandleTypeDef *hspi)
{
    GPIO_InitTypeDef GPIO_InitStruct;
    /* Enable GPIO TX/RX clock */
    SPIx_SCK_GPIO_CLK_ENABLE();
    SPIx_MISO_GPIO_CLK_ENABLE();
    SPIx_MOSI_GPIO_CLK_ENABLE();
    /* Enable SPI clock */
    SPIx_CLK_ENABLE();
}

```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 41. oldal
--	--	---

```

/* SPI SCK GPIO pin configuration */
GPIO_InitStruct.Pin      = SPIx_SCK_PIN;
GPIO_InitStruct.Mode     = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull     = GPIO_PULLUP;
GPIO_InitStruct.Speed    = GPIO_SPEED_FAST;
GPIO_InitStruct.Alternate = SPIx_SCK_AF;

HAL_GPIO_Init(SPIx_SCK_GPIO_PORT, &GPIO_InitStruct);

/* SPI MISO GPIO pin configuration */
GPIO_InitStruct.Pin = SPIx_MISO_PIN;
GPIO_InitStruct.Alternate = SPIx_MISO_AF;

HAL_GPIO_Init(SPIx_MISO_GPIO_PORT, &GPIO_InitStruct);

/* SPI MOSI GPIO pin configuration */
GPIO_InitStruct.Pin = SPIx_MOSI_PIN;
GPIO_InitStruct.Alternate = SPIx_MOSI_AF;

HAL_GPIO_Init(SPIx_MOSI_GPIO_PORT, &GPIO_InitStruct);
}

```

Az alkalmazás main függvényében kerül megvalósításra az SPI periféria inicializálása, valamint az adatküldést és fogadást megvalósító alkalmazáslogika.

A HAL Library, a LED-ek és a mikrokontroller órajelének inicializálása után az SPI perifériát konfiguráljuk. A konfigurálás ugyanolyan szisztéma szerint zajlik, mint ahogy azt az időzítőknél is láthattuk. Egy leíró struktúrát kell kitölteni, melynek Instance tagjában meg kell adni, hogy a beállítások mely SPI perifériára vonatkozzanak, majd az Init alstruktúra tagjainak beállításával specifikálhatjuk a kívánt működési módot (pl. SPI Master/Slave, SCK frekvencia, bitsorrend, adatbitek száma, stb.). Az inicializációs struktúrában leírt beállításokat a HAL_SPI_Init függvény meghívásával tudjuk végrehajtani. Amennyiben az inicializálás sikertelen lenne – mert pl. egymásnak ellentmondó konfigurációs beállításokat használtunk, akkor egy hibakezelő ágba jutunk, ahol egy általános Error_Handler hibakezelő függvény meghívásával egy végtelen ciklusban megáll az alkalmazásunk.

```

SPI_HandleTypeDef SpiHandle;

int main(void)
{
    HAL_Init();
    /* Configure LED3, LED4, LED5 and LED6 */
    BSP_LED_Init(LED3);
    BSP_LED_Init(LED4);
    BSP_LED_Init(LED5);
    BSP_LED_Init(LED6);
    /* Configure the system clock to 168 MHz */
    SystemClock_Config();

    /* Set the SPI parameters */
    SpiHandle.Instance      = SPIx;
    SpiHandle.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_256;
}

```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 42. oldal
--	--	---

```

SpiHandle.Init.Direction      = SPI_DIRECTION_2LINES;
SpiHandle.Init.CLKPhase       = SPI_PHASE_1EDGE;
SpiHandle.Init.CLKPolarity    = SPI_POLARITY_HIGH;
SpiHandle.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
SpiHandle.Init.CRCPolynomial  = 7;
SpiHandle.Init.DataSize       = SPI_DATASIZE_8BIT;
SpiHandle.Init.FirstBit       = SPI_FIRSTBIT_MSB;
SpiHandle.Init.NSS            = SPI_NSS_SOFT;
SpiHandle.Init.TIMode         = SPI_TIMODE_DISABLE;
#ifdef MASTER_BOARD
    SpiHandle.Init.Mode = SPI_MODE_MASTER;
#else
    SpiHandle.Init.Mode = SPI_MODE_SLAVE;
#endif /* MASTER_BOARD */
if (HAL_SPI_Init(&SpiHandle) != HAL_OK)
{
    /* Initialization Error */
    Error_Handler();
}

```

Az SPI periféria inicializálását követően az alkalmazáslogika megvalósítása következik. Amennyiben a forráskódot az SPI master kártyára szeretnénk lefordítani, akkor figyelniünk kell a nyomógomb állapotát – mivel a master kártyán történő gombnyomásra fog elindulni az SPI kommunikáció.

```

#ifdef MASTER_BOARD
    /* Configure USER Button */
    BSP_PB_Init(BUTTON_KEY, BUTTON_MODE_GPIO);
    /* Wait for USER Button press before starting the Communication */
    while (BSP_PB_GetState(BUTTON_KEY) != 1)
    {
        BSP_LED_Toggle(LED3);
        HAL_Delay(40);
    }
    BSP_LED_Off(LED3);
#endif /* MASTER_BOARD */

```

A kommunikációt a master kártya fogja indítani, méghozzá a HAL_SPI_TransmitReceive függvény meghívásával. A függvény paramétereiben meg kell adnunk az SPI perifériánkat azonosító struktúrára mutató pointert, a küldendő adatokat tartalmazó buffer címét, a bejövő adatok tárolására használt buffer címét, a bufferek méretét (a kommunikáció jellegéből fakadóan a bufferek mérete egyforma kell hogy legyen), illetve egy timeout értéket. A megadott timeout érték azt határozza meg, hogy az SPI periféria hány ms ideig várákozzon a sikeres küldést jelző flag-re.

A HAL_SPI_TransmitReceive függvény implementációja olyan, hogy egy while ciklusban bájtonként küldi és fogadja az adatokat, aztán várákozik a sikeres küldést/fogadást jelző flag-re – tehát a hívó kódrészlet futását mindaddig blokkolja amíg a küldés/fogadás sikeresen le nem zajlott, illetve a timeout le nem járt. Ez kifejezetten előnytelen, hiszen közben csinálhatna más is – pl. előkészíthetné a következő küldendő adatokat stb.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 43. oldal
--	--	---

A main függvény további részében a küldő-fogadó függvény visszatérési értékének függvényében elvégezzük az adatok összehasonlítását, illetve hibakezelő ágakat hajtunk végre.

```

switch(HAL_SPI_TransmitReceive(&SpiHandle,
    (uint8_t*)aTxBuffer,
    (uint8_t *)aRxBuffer, BUFFERSIZE, 5000))
{
case HAL_OK:
    /* Communication is completed,
    Compare the sent and received buffers */
    if(Buffercmp((uint8_t*)aTxBuffer,
        (uint8_t*)aRxBuffer, BUFFERSIZE))
    {
        /* Transfer error in transmission process */
        Error_Handler();
    }
    /* Turn LED4 on: Transfer in transmission process is correct */
    BSP_LED_On(LED4);
    /* Turn LED6 on: Transfer in reception process is correct */
    BSP_LED_On(LED6);
    break;
case HAL_TIMEOUT:
    /* A Timeout occurred Call Timeout Handler */
    Timeout_Error_Handler();
    break;
    /* An Error occurred */
case HAL_ERROR:
    /* Call Timeout Handler */
    Error_Handler();
    break;
default:
    break;
}
while(1);
}

```

Amennyiben a küldés-fogadás ideje alatt nem szeretnénk, hogy az alkalmazásunk az SPI kommunikáció sikerességére várjon, hanem csináljon esetleg mást is, akkor két lehetséges megoldást használhatunk. Az egyik megoldás, hogy megszakításon keresztül kérünk jelzést az SPI perifériától, ha hiba történt, illetve sikeres volt a kommunikáció. Ez a megoldás nem túl bonyolult, egyszerűen csak másik függvényt kell meghívunk a kommunikációhoz. A HAL_SPI_TransmitReceive_IT függvény az SPI periféria kommunikációhoz szükséges regisztereit megfelelően beállítja, és a megszakításokat is engedélyezi, majd visszatér. A függvény visszatérése után gyakorlatilag bármit csinálhat a programunk – az egyetlen kitétel, hogy amíg a periféria nem jelezte megszakítással a sikeres kommunikációt ill. hibát, addig az SPI periférián nem kezdeményezhetünk újabb kommunikációt.

```

if(HAL_SPI_TransmitReceive_IT(&SpiHandle, (uint8_t*)aTxBuffer,
    (uint8_t *)aRxBuffer, BUFFERSIZE) != HAL_OK)
{

```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 44. oldal
--	--	---

```

/* Transfer error in transmission process */
Error_Handler();
}
/* Do anything, but do not try to communicate through the SPI channel
*/
while (HAL_SPI_GetState(&SpiHandle) != HAL_SPI_STATE_READY)
{
}
if(Buffercmp((uint8_t*)aTxBuffer, (uint8_t*)aRxBuffer, BUFFERSIZE))
{
/* Transfer error in transmission process */
Error_Handler();
}

```

A periféria a sikeres kommunikációt, illetve hibát megszakítással fogja jelezni, tehát meg kell írunk a natív megszakításkezelő rutint, amiben a HAL Library megszakításkezelőjét kell meghívni. Ezen kívül természetesen a callback függvényeket is meg kell írunk az egyes események alkalmazásspecifikus kezeléséhez – ezt láthatjuk az alábbi kódrészletekben.

```

void SPIx_IRQHandler(void)
{
HAL_SPI_IRQHandler(&SpiHandle);
}

void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi)
{
/* Turn LED4 on: Transfer in transmission process is correct */
BSP_LED_On(LED4);
/* Turn LED6 on: Transfer in reception process is correct */
BSP_LED_On(LED6);
}

void HAL_SPI_ErrorCallback(SPI_HandleTypeDef *hspi)
{
/* Turn LED5 on: Transfer error in reception/transmission process */
BSP_LED_On(LED5);
}

```

A megszakításokat használó megoldásunk akkor hatékony, ha nem nagy mennyiségű adatot szeretnénk átküldeni az SPI csatornán. Ha nagyobb adatmennyiséget szeretnénk küldeni-fogadni, akkor megfontolandó, hogy a mikrokontrollerbe integrált DMA kontroller segítségét igénybe vegyük.

A DMA inicializálásához a már egyéb perifériáknál is megszokott inicializáló struktúrát kell kitölteni. Az Instance tagban meg kell adni, hogy melyik DMA vezérlőre vonatkozzanak a beállításaink, majd az Init alstruktúrát kell a már megszokottak szerint kitölteni. Meg kell adnunk, hogy melyik streamet használja, milyen irányú a kommunikáció, milyen adatmennyiséget továbbítson egy lépésben, és hogy legyen-e címléptetés a küldő ill. fogadó oldalon. A HAL_DMA_Init függvénnyel alkalmazzuk a konfigurációt a DMA vezérlőre. Ezek után a __HAL_LINKDMA makróval hozzárendeljük az SPI perifériánkat a DMA streamhez.

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 45. oldal
--	--	---

```
static DMA_HandleTypeDef hdma_tx;
hdma_tx.Instance = SPIx_TX_DMA_STREAM;

hdma_tx.Init.Channel = SPIx_TX_DMA_CHANNEL;
hdma_tx.Init.Direction = DMA_MEMORY_TO_PERIPH;
hdma_tx.Init.PeriphInc = DMA_PINC_DISABLE;
hdma_tx.Init.MemInc = DMA_MINC_ENABLE;
hdma_tx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
hdma_tx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
hdma_tx.Init.Mode = DMA_NORMAL;
hdma_tx.Init.Priority = DMA_PRIORITY_LOW;
hdma_tx.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
hdma_tx.Init.FIFOThreshold = DMA_FIFO_THRESHOLD_FULL;
hdma_tx.Init.MemBurst = DMA_MBURST_INC4;
hdma_tx.Init.PeriphBurst = DMA_PBURST_INC4;

HAL_DMA_Init(&hdma_tx);

/* Associate the initialized DMA handle to the the SPI handle */
__HAL_LINKDMA(hspi, hdmatx, hdma_tx);
```

Mivel kétirányú kommunikációt szeretnénk megvalósítani, így nem csak az SPI küldéshez, hanem a fogadáshoz is létre kell hoznunk egy DMA streamet. A küldő részhez képest annyi az eltérés, hogy a kommunikáció irányát kell fordítva beállítani, ill. a címléptetést is a másik végpontnál kell engedélyezni.

```
static DMA_HandleTypeDef hdma_rx;
hdma_rx.Instance = SPIx_RX_DMA_STREAM;

hdma_rx.Init.Channel = SPIx_RX_DMA_CHANNEL;
hdma_rx.Init.Direction = DMA_PERIPH_TO_MEMORY;
hdma_rx.Init.PeriphInc = DMA_PINC_DISABLE;
hdma_rx.Init.MemInc = DMA_MINC_ENABLE;
hdma_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
hdma_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
hdma_rx.Init.Mode = DMA_NORMAL;
hdma_rx.Init.Priority = DMA_PRIORITY_HIGH;
hdma_rx.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
hdma_rx.Init.FIFOThreshold = DMA_FIFO_THRESHOLD_FULL;
hdma_rx.Init.MemBurst = DMA_MBURST_INC4;
hdma_rx.Init.PeriphBurst = DMA_PBURST_INC4;
HAL_DMA_Init(&hdma_rx);

/* Associate the initialized DMA handle to the the SPI handle */
__HAL_LINKDMA(hspi, hdmarx, hdma_rx);
```

A DMA vezérlő megszakítással fogja jelezni a kommunikáció végét, tehát ezeknek a megszakításoknak a prioritását be kell állítanunk, és engedélyeznünk kell azokat.

```
/* NVIC configuration for DMA transfer complete interrupt */

HAL_NVIC_SetPriority(SPIx_DMA_TX_IRQn, 0, 1);
HAL_NVIC_EnableIRQ(SPIx_DMA_TX_IRQn);
```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 46. oldal
--	--	---

```

/* NVIC configuration for DMA transfer complete interrupt */
HAL_NVIC_SetPriority(SPIx_DMA_RX_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(SPIx_DMA_RX_IRQn);

HAL_NVIC_SetPriority(SPIx_IRQn, 0, 2);
HAL_NVIC_EnableIRQ(SPIx_IRQn);

```

A kommunikációt a HAL_SPI_TransmitReceive_DMA függvénnyel kezdeményezhetjük. A függvény visszatérése után az alkalmazásunk bármit csinálhat – de nem kezdeményezhetünk újabb SPI kommunikációt.

```

if(HAL_SPI_TransmitReceive_DMA(&SpiHandle, (uint8_t*)aTxBuffer,
(uint8_t *)aRxBuffer, BUFFERSIZE) != HAL_OK)
{
    /* Transfer error in transmission process */
    Error_Handler();
}
while (HAL_SPI_GetState(&SpiHandle) != HAL_SPI_STATE_READY)
{
}
if(Buffercmp((uint8_t*)aTxBuffer, (uint8_t*)aRxBuffer, BUFFERSIZE))
{
    /* Transfer error in transmission process */
    Error_Handler();
}

```

2.20.7 Analóg-digitális átalakító használata - Mintapélda

Az utolsó mintapéldánkban a mikrokontrollerbe integrált AD átalakítót használjuk úgy, hogy a konvertált mérési adatok a DMA vezérlő segítségével jutnak el a memóriába.

```

void HAL_ADC_MspInit(ADC_HandleTypeDef* hadc)
{
    GPIO_InitTypeDef          GPIO_InitStruct;
    static DMA_HandleTypeDef  hdma_adc;

    /* Enable GPIO clock */
    ADCx_CHANNEL_GPIO_CLK_ENABLE();
    /* ADC3 Periph clock enable */
    ADCx_CLK_ENABLE();
    /* Enable DMA2 clock */
    DMAx_CLK_ENABLE();

    /* ADC3 Channel8 GPIO pin configuration */
    GPIO_InitStruct.Pin = ADCx_CHANNEL_PIN;
    GPIO_InitStruct.Mode = GPIO_MODE_ANALOG;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(ADCx_CHANNEL_GPIO_PORT, &GPIO_InitStruct);
    hdma_adc.Instance = ADCx_DMA_STREAM;
    hdma_adc.Init.Channel = ADCx_DMA_CHANNEL;
    hdma_adc.Init.Direction = DMA_PERIPH_TO_MEMORY;
    hdma_adc.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_adc.Init.MemInc = DMA_MINC_ENABLE;

```

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 47. oldal
--	--	---

```

hdma_adc.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;
hdma_adc.Init.MemDataAlignment = DMA_MDATAALIGN_WORD;
hdma_adc.Init.Mode = DMA_CIRCULAR;
hdma_adc.Init.Priority = DMA_PRIORITY_HIGH;
hdma_adc.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
hdma_adc.Init.FIFOThreshold = DMA_FIFO_THRESHOLD_HALFFULL;
hdma_adc.Init.MemBurst = DMA_MBURST_SINGLE;
hdma_adc.Init.PeriphBurst = DMA_PBURST_SINGLE;
HAL_DMA_Init(&hdma_adc);

/* Associate the initialized DMA handle to the the ADC handle */
HAL_LINKDMA(hadc, DMA_Handle, hdma_adc);
/* NVIC configuration for DMA transfer complete interrupt */
HAL_NVIC_SetPriority(ADCx_DMA_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(ADCx_DMA_IRQn);
}

```

A DMA vezérlő inicializálása szinte teljesen megegyezik az előző mintapéldában alkalmazott beállításokkal.

Az alkalmazásunk logikája is roppant egyszerű, inicializáljuk a perifériáinkat, majd pedig elindítjuk a konverziót. A konverzió eredményét a DMA vezérlő egy globális változóban tárolja el. Amint a DMA vezérlő letette a konvertált értéket a globális változóba, az AD konverter jelezni fog egy megszakítással, hogy a konverzió véget ért.

Az AD konverter beállításainál meg kell adnunk, hogy milyen órajelfrekvenciával működjön a konverzió, milyen formában, ill. milyen felbontással tárolja az eredményt.

```

int main(void)
{
    ADC_ChannelConfTypeDef sConfig;
    HAL_Init();
    /* Configure LED4 and LED5 */
    BSP_LED_Init(LED4);
    BSP_LED_Init(LED5);

    /*Configure the ADC peripheral */
    AdcHandle.Instance = ADCx;

    AdcHandle.Init.ClockPrescaler = ADC_CLOCKPRESCALER_PCLK_DIV2;
    AdcHandle.Init.Resolution = ADC_RESOLUTION_12B;
    AdcHandle.Init.ScanConvMode = DISABLE;
    AdcHandle.Init.ContinuousConvMode = ENABLE;
    AdcHandle.Init.DiscontinuousConvMode = DISABLE;
    AdcHandle.Init.NbrOfDiscConversion = 0;
    AdcHandle.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
    AdcHandle.Init.ExternalTrigConv = ADC_EXTERNALTRIGCONV_T1_CC1;
    AdcHandle.Init.DataAlign = ADC_DATAALIGN_RIGHT;
    AdcHandle.Init.NbrOfConversion = 1;
    AdcHandle.Init.DMAContinuousRequests = ENABLE;
    AdcHandle.Init.EOCSelection = DISABLE;
    if (HAL_ADC_Init(&AdcHandle) != HAL_OK) Error_Handler();
}

```

Az eredmény tárolásának helyét és a mintavételezési időt az AD csatorna konfigurációs struktúrájának kitöltésével tudjuk beállítani. A csatorna konfigurálását a

BME Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék	Mikrokontroller alapú rendszerek előadás 2. fejezet	MAR_EA_2c.DOCX 2023.11.24. Szabó Zoltán II / 48. oldal
--	--	---

HAL_ADC_ConfigChannel függvénnyel végezhetjük el. Ezek után a HAL_ADC_Start_DMA függvény meghívásával indul a konverzió. A konverzió végét pedig a már korábban is említett megszakításon keresztül jelzi majd az AD konverter.

```

sConfig.Channel = ADCx_CHANNEL;
sConfig.Rank = 1;
sConfig.SamplingTime = ADC_SAMPLETIME_28CYCLES;
sConfig.Offset = 0;

if(HAL_ADC_ConfigChannel(&AdcHandle, &sConfig) != HAL_OK)
{
    /* Channel Configuration Error */
    Error_Handler();
}
if(HAL_ADC_Start_DMA(&AdcHandle,
    (uint32_t*)&uhADCxConvertedValue, 1) != HAL_OK)
{
    /* Start Conversation Error */
    Error_Handler();
}
while(1);
}
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* AdcHandle)
{
    /* Turn LED4 on: Transfer process is correct */
    BSP_LED_On(LED4);
}

```